

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403

anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450

phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông

Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhẫn

Chữ ký của GVHD

KHOA: (Điền tên khoa)

Hà Nội, 01/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng <...> năm <...>

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	4
DANH SÁCH HÌNH VẼ	6
DANH SÁCH BẢNG	7
PHÂN CHIA CÔNG VIỆC	8
1 TỔNG QUAN ĐỀ TÀI	9
1.1 Bối cảnh và lý do chọn đề tài	9
1.2 Mục tiêu của đề án	10
1.2.1 Mục tiêu tổng quát	10
1.2.2 Mục tiêu cụ thể	10
1.3 Phạm vi và giới hạn	11
1.3.1 Phạm vi nghiên cứu và triển khai	11
1.3.2 Giới hạn của đề tài	11
1.4 Phương pháp thực hiện	11
1.5 Đóng góp chính của đề án	12
1.6 Cấu trúc báo cáo	12
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN	14
2.1 Tổng quan IoT và Smart Building	14
2.2 Zigbee và IEEE 802.15.4	15
2.3 Vai trò thiết bị trong mạng Zigbee	15
2.4 Topology mạng Zigbee	16
2.5 Zigbee Protocol Stack	17
2.6 Zigbee Cluster Library và mô hình thiết bị	18
2.7 Gateway trong hệ thống IoT	19
2.8 MQTT và cơ chế Publish/Subscribe	19
2.9 UART, ASH và EZSP giữa Host và EFR32 NCP	20
2.10 REST API, Cloud Backend và Database	21
2.11 Mobile App và State Management	22
2.12 Firmware, Bootloader và OTA	22
2.13 Security trong Zigbee và IoT	23
2.14 Công nghệ sử dụng trong đề án	23
3 PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	25
3.1 Phạm vi và giả định thiết kế	25
3.2 Phân tích yêu cầu hệ thống	26
3.2.1 Tác nhân sử dụng hệ thống	26
3.2.2 Yêu cầu chức năng	26
3.2.3 Yêu cầu phi chức năng	27
3.3 Kiến trúc tổng thể hệ thống	27
3.4 Thiết kế thiết bị và trạng thái	28

3.4.1	Device model	28
3.4.2	State, event và command	28
3.5	Luồng dữ liệu chính	29
3.5.1	Luồng uplink: thiết bị báo cáo lên Cloud	29
3.5.2	Luồng downlink: người dùng gửi lệnh điều khiển	29
3.5.3	Luồng automation	29
3.6	Thiết kế MQTT contract	30
3.7	Thiết kế Cloud Backend và cơ sở dữ liệu	31
3.7.1	REST API	31
3.7.2	Cơ sở dữ liệu	32
3.8	Thiết kế Gateway	32
3.9	Thiết kế Mobile App	33
3.10	Các quyết định thiết kế và trade-off	33
4	TRIỂN KHAI HỆ THỐNG	35
4.1	Môi trường triển khai và cấu trúc mã nguồn	35
4.2	Triển khai Firmware cho thiết bị Zigbee	35
4.2.1	Triển khai Coordinator / NCP Board	36
4.2.2	Triển khai Router đèn (Z3Light Device)	36
4.2.3	Triển khai Router công tắc (Z3Switch Device)	36
4.2.4	Triển khai cảm biến hiện diện (Occupancy Sensor)	36
4.3	Triển khai Gateway Service nhúng	36
4.3.1	MQTT client và command queue	36
4.3.2	Command handler và điều khiển thiết bị	37
4.3.3	Discovery, registry và rule cục bộ	37
4.4	Triển khai Cloud Backend	37
4.4.1	FastAPI app lifecycle và router	37
4.4.2	Database model và command lifecycle	37
4.4.3	Cấu hình biến môi trường	38
4.4.4	MQTT service và timeout worker	38
4.5	Triển khai Mobile App	38
4.6	Triển khai Deployment	38
4.7	Luồng vận hành End-to-End Command	39
4.8	Giới hạn triển khai OTA trong phiên bản hiện tại	39
4.9	Tổng kết chương	39
5	KIỂM THỬ VÀ ĐÁNH GIÁ	40
5.1	Kế hoạch kiểm thử	40
5.2	Môi trường và cấu hình kiểm thử	40
5.2.1	Cấu hình phần cứng thử nghiệm	40
5.3	Kết quả thực thi các kịch bản kiểm thử	41
5.3.1	Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị	41
5.3.2	Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển	42
5.3.3	Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy	43
5.4	Đánh giá kết quả kiểm thử	44
5.4.1	Mức độ hoàn thành các tiêu chí hệ thống	44
5.4.2	Phân tích độ trễ của hệ thống	45
5.5	Hạn chế của hệ thống phát hiện qua kiểm thử	45

6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
6.1	Kết luận chung	47
6.2	Kiến thức và kỹ năng đạt được	47
6.3	Hướng phát triển tiếp theo	48
6.3.1	Hoàn thiện phần cứng và mạng thiết bị	48
6.3.2	Nâng cấp tính năng cho Gateway nhúng	48
6.3.3	Nâng cấp nền tảng Cloud và Ứng dụng di động	48
6.3.4	Tăng cường An ninh bảo mật	49
	TÀI LIỆU THAM KHẢO	50

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-The-Air (firmware update)
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1	Quy trình nghiên cứu và triển khai hệ thống	11
2.1	Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building	14
2.2	Minh họa topology dạng mesh trong mạng Zigbee	17
2.3	Kiến trúc tầng giao thức Zigbee	18
2.4	Mô hình broker trung tâm trong MQTT	20
2.5	Minh họa giao tiếp UART giữa hai thành phần phần cứng	21
3.1	Kiến trúc runtime của hệ thống Smart Building trong đồ án	27
3.2	Luồng uplink đồng bộ trạng thái và sự kiện	29
3.3	Luồng downlink và phản hồi command	29
3.4	Ranh giới thiết kế automation trong phiên bản hiện tại	30

DANH SÁCH BẢNG

2	Bảng phân chia công việc	8
1.1	Đóng góp chính và ý nghĩa kỹ thuật của đề án	12
2.1	Đặc điểm chính của Zigbee trong hệ thống Smart Building	15
2.2	Vai trò thiết bị trong mạng Zigbee	16
2.3	Một số cluster liên quan đến đề tài	18
2.4	Vai trò của các thành phần Cloud trong hệ thống	22
2.5	Tóm tắt công nghệ nền được sử dụng trong đề án	24
3.1	Phạm vi thiết kế trong đề án	25
3.2	Các tác nhân chính trong hệ thống	26
3.3	Các yêu cầu chức năng chính	26
3.4	Các yêu cầu phi chức năng	27
3.5	Trách nhiệm của các khối chính	28
3.6	Ánh xạ loại thiết bị và chức năng runtime	28
3.7	Các nhóm topic MQTT chính	30
3.8	Các nhóm REST API chính	32
3.9	Các bảng dữ liệu chính	32
3.10	Các module logic chính trong Gateway	33
3.11	Các trade-off chính trong thiết kế	34
4.1	Các thành phần triển khai chính trong mã nguồn	35
4.2	Các biến môi trường cấu hình chính của Cloud Backend	38
5.1	Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm	41
5.2	Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)	42
5.3	Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)	42
5.4	Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)	43
5.5	Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống	45

PHÂN CHIA CÔNG VIỆC

STT	Công việc	Sinh viên thực hiện
Firmware – Zigbee Devices		
1	<Firmware Coordinator: tạo mạng, Trust Center, ZCL, UART bridge>	<SV>
2	<Firmware Light Device: On/Off Server, LED indicator, state report>	<SV>
3	<Firmware Switch Device: On/Off Client, binding, button control>	<SV>
4	<Firmware Occupancy Sensor: occupancy detection, ZCL report>	<SV>
Gateway Service (Ubuntu / Raspberry Pi)		
5	<Gateway UART module: kết nối COM, đọc line, reconnect>	<SV>
6	<Gateway MQTT bridge: subscribe cmd, publish state/telemetry/ack>	<SV>
7	<Gateway rules engine, ACK routing, retry/timeout>	<SV>
8	<Gateway Admin API (/health, /status, /config, /rules)>	<SV>
Cloud – Firebase		
9	<Firebase setup: Realtime DB / Firestore schema, Cloud Functions>	<SV>
10	<MQTT – Firebase bridge: device state sync, command relay>	<SV>
11	<Event history, logging, device management trên cloud>	<SV>
Mobile App – Flutter		
12	<Flutter app: device list, light on/off control UI>	<SV>
13	<Flutter app: occupancy state display, realtime update>	<SV>
14	<Flutter app: event history, notification>	<SV>
Tổng hợp & Tài liệu		
15	<Tổng hợp và hoàn thiện báo cáo>	<SV 1, SV 2>

Bảng 2: Bảng phân chia công việc

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài xây dựng hệ thống IoT Zigbee Smart Building. Tiếp theo, chương xác định rõ mục tiêu tổng quát, mục tiêu cụ thể, phạm vi nghiên cứu và các giới hạn thực tế của đề án. Cuối cùng, phương pháp thực hiện, những đóng góp chính về mặt kỹ thuật và cấu trúc tổng thể của báo cáo cũng được khái quát hóa nhằm cung cấp cái nhìn toàn cảnh trước khi đi vào các nội dung kỹ thuật chi tiết.

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ mạng không dây và Internet vạn vật (Internet of Things - IoT) đã thúc đẩy việc ứng dụng tự động hóa vào quản lý vận hành tòa nhà. Smart Building (tòa nhà thông minh) không còn là khái niệm lý thuyết mà đã trở thành xu hướng tất yếu nhằm tối ưu hóa lượng điện năng tiêu thụ, nâng cao độ an toàn và mang lại trải nghiệm tiện nghi cho người sử dụng. Trong các hệ thống này, nhu cầu giám sát trạng thái môi trường, phát hiện sự hiện diện của con người và điều khiển các thiết bị phụ tải (như đèn chiếu sáng, công tắc) là những bài toán cốt lõi cần giải quyết.

Đối với mạng kết nối nội bộ (local network) trong tòa nhà, việc sử dụng các công nghệ truyền thông không dây phổ biến như Wi-Fi thường bộc lộ những hạn chế lớn về mặt tiêu thụ năng lượng và khả năng chịu tải của điểm truy cập trung tâm khi số lượng thiết bị lên tới hàng trăm node. Do đó, các công thức truyền thông tiết kiệm năng lượng, hỗ trợ cấu trúc liên kết mạng lưới (mesh network) như Zigbee được xem là giải pháp tối ưu. Zigbee hoạt động trên tiêu chuẩn IEEE 802.15.4, cho phép các thiết bị tự động định tuyến gói tin qua nhau, giúp mở rộng phạm vi phủ sóng mà không yêu cầu công suất phát lớn, đồng thời giảm thiểu đáng kể chi phí duy trì nguồn pin cho các cảm biến.

Tuy nhiên, các mạng Zigbee cục bộ không thể tự kết nối trực tiếp với các ứng dụng di động hoặc hệ thống quản lý từ xa qua mạng IP diện rộng. Điều này đòi hỏi sự xuất hiện của một thiết bị trung gian gọi là Gateway. Thiết bị Gateway đóng vai trò làm cầu nối chuyển đổi giữa giao thức Zigbee RF tầm ngắn và mạng IP (chạy các giao thức như MQTT, HTTP) kết nối lên Cloud. Sự phối hợp giữa Gateway và hệ thống đám mây (Cloud Backend) cho phép lưu trữ lịch sử vận hành, xử lý các kịch bản tự động hóa phức tạp và cung cấp các giao diện lập trình ứng dụng (API) cho ứng dụng di động của người dùng cuối.

Hơn nữa, các giải pháp smart building thương mại hiện nay thường là các hệ sinh thái đóng (closed ecosystem) của các nhà sản xuất lớn, gây khó khăn cho việc tích hợp thiết bị của bên thứ ba và hạn chế khả năng tùy biến logic điều khiển. Xuất phát từ nhu cầu nghiên cứu một nền tảng quản lý thiết bị mở, có khả năng làm chủ từ firmware thiết bị, logic xử lý tại gateway cho đến kiến trúc cloud backend, đề tài "**Hệ thống IoT Zigbee Smart Building**" đã được lựa chọn để thực hiện. Đề án tập trung nghiên cứu thiết kế và tích hợp đồng bộ các lớp thiết bị, gateway và dịch vụ đám mây nhằm đảm bảo luồng dữ liệu hai chiều nhanh chóng, tin cậy và có khả năng mở rộng.

1.2 Mục tiêu của đề án

Để giải quyết các bài toán đặt ra trong bối cảnh trên, đề án xác định rõ mục tiêu tổng quát và các mục tiêu cụ thể cần đạt được.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là thiết kế, chế tạo và tích hợp hoàn chỉnh một hệ thống IoT quản lý Smart Building dựa trên chuẩn truyền thông không dây Zigbee, kết nối qua Gateway C tới hệ thống Cloud Backend, hỗ trợ giám sát trạng thái thiết bị thực tế gần thời gian thực và tự động hóa điều khiển phụ tải.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát, đề án đề ra các mục tiêu cụ thể cho từng thành phần trong hệ thống:

Phía thiết bị cuối (End Devices): Thiết kế và nạp firmware cho các thiết bị Zigbee 3.0 dựa trên vi điều khiển EFR32MG12 bao gồm thiết bị đèn (Z3Light), công tắc (Z3Switch) và cảm biến hiện diện (Occupancy/Motion Sensor) hoạt động ổn định trong mạng mesh Zigbee.

Phía Gateway trung gian: Triển khai ứng dụng Z3Gateway dạng tiến trình đơn (single-process) viết bằng ngôn ngữ C chạy trên Linux host. Tích hợp trực tiếp thư viện kết nối MQTT để giao tiếp trực tiếp với Cloud, loại bỏ hoàn toàn các lớp trung gian trung chuyển IPC phức tạp nhằm giảm độ trễ và tăng tính ổn định.

Phía dịch vụ đám mây (Cloud Backend): Xây dựng Cloud Backend sử dụng framework FastAPI (Python 3.12) kết nối cơ sở dữ liệu PostgreSQL. Hệ thống đám mây phải tiếp nhận các bản tin telemetry báo cáo trạng thái, quản lý vòng đời lệnh điều khiển, lưu trữ lịch sử sự kiện và cung cấp hệ thống REST API chuẩn hóa.

Phía ứng dụng giám sát (Dashboard/App): Thiết kế các giao diện mô phỏng dưới dạng API hoặc màn hình quản lý cho phép theo dõi trực quan danh sách thiết bị, trạng thái vận hành hiện tại và gửi lệnh bật/tắt thiết bị đèn.

Về mặt kiểm thử và đánh giá: Xây dựng bộ kịch bản kiểm thử (test cases) end-to-end hoàn chỉnh nhằm đo đặc độ trễ phản hồi của lệnh điều khiển, kiểm chứng tính chính xác của cơ chế đồng bộ trạng thái và ghi nhận bằng chứng vận hành (logs, traces).

1.3 Phạm vi và giới hạn

Việc xác định phạm vi nghiên cứu và giới hạn thực thi giúp đồ án tập trung vào các vấn đề kỹ thuật trọng tâm, tránh phân tán nguồn lực vào các tính năng thương mại phức tạp.

1.3.1 Phạm vi nghiên cứu và triển khai

Đồ án tập trung thực hiện các nội dung sau:

Thiết lập mạng Zigbee cục bộ: Sử dụng một Coordinator chạy firmware NCP (Network Co-Processor) kết nối qua UART/ASH với Gateway host.

Triển khai kịch bản gia nhập mạng: Thực hiện permit-join, đồng bộ trạng thái thực tế lên cloud (reported state) và truyền đạt lệnh trạng thái mong muốn (desired state) qua giao thức MQTT.

Đồng bộ trạng thái thiết bị và kịch bản tự động hóa cục bộ: Hỗ trợ local automation cơ bản thông qua cơ chế APS Binding trực tiếp giữa công tắc và đèn trong mạng Zigbee mesh.

Theo dõi vòng đời của lệnh điều khiển: Theo dõi chặt chẽ quá trình xử lý lệnh từ Cloud xuống thiết bị qua các pha trạng thái rõ ràng (accepted → queued → sent → executed/failed/-timeout).

Triển khai hệ thống Cloud Backend: Deploy hệ thống trên hạ tầng điện toán đám mây dưới dạng các container Docker được đóng gói và cấu hình hoàn chỉnh.

1.3.2 Giới hạn của đề tài

Do hạn chế về mặt thời gian và trang thiết bị phần cứng, đồ án tồn tại một số giới hạn như sau:

Quy mô thử nghiệm: Do hạn chế về mặt thời gian và trang thiết bị phần cứng, hệ thống chỉ dừng lại ở quy mô thử nghiệm phòng lab (lab-scale) với số lượng thiết bị demo hạn chế (khoảng 3-5 node), chưa được triển khai thực tế trên quy mô tòa nhà nhiều tầng với mật độ cản trở vật lý cao.

Ứng dụng di động: Ứng dụng di động Flutter và cơ chế điều khiển tích hợp qua giao thức Bluetooth Low Energy (BLE) chỉ dừng lại ở mức thiết kế giao diện tĩnh (placeholders) hoặc mô phỏng logic client, chưa được tích hợp kết nối API thực tế chạy trên thiết bị di động thật trong bản v1 này.

Tính năng nâng cao: Các tính năng nâng cao như cơ chế cập nhật firmware từ xa (OTA Update), bảo mật mã hóa đường truyền MQTT qua TLS (port 8883) và hệ thống xác thực người dùng (Authentication API) chỉ được thực hiện ở mức phân tích thiết kế kịch bản lý thuyết (contracts), chưa có mã nguồn triển khai chạy thực tế.

1.4 Phương pháp thực hiện

Để thực hiện đồ án một cách khoa học, quy trình nghiên cứu và phát triển được chia làm 5 bước tuần tự như được mô tả trong Hình 1.1.

Hình 1.1: Quy trình nghiên cứu và triển khai hệ thống

Quy trình thực hiện bao gồm các giai đoạn cụ thể sau:

Giai đoạn 1 – Khảo sát và nghiên cứu công nghệ: Nghiên cứu tài liệu kỹ thuật về chuẩn Zigbee 3.0, thư viện ZCL (Zigbee Cluster Library) của Silicon Labs, giao thức truyền thông

MQTT và kiến trúc API hướng tài nguyên sử dụng REST.

Giai đoạn 2 – Thiết kế hệ thống: Xác định sơ đồ khối kiến trúc hệ thống 4 lớp, thiết kế cấu trúc cơ sở dữ liệu PostgreSQL, xây dựng hợp đồng trao đổi bản tin MQTT (MQTT Contract JSON Schema) và cấu hình các topic phân cấp.

Giai đoạn 3 – Triển khai chi tiết từng module: Lập trình firmware cho các SoC EFR32MG12; viết mã nguồn Z3Gateway bằng ngôn ngữ C; xây dựng Cloud API bằng FastAPI.

Giai đoạn 4 – Tích hợp hệ thống: Kết nối Gateway host với chip NCP qua UART, kết nối Gateway lên Mosquitto Broker chạy trên container Docker, thiết lập liên kết đồng bộ giữa cơ sở dữ liệu và MQTT Client chạy nền trên Cloud.

Giai đoạn 5 – Kiểm thử và đánh giá: Chạy các kịch bản kiểm thử tích hợp tự động bằng pytest cho backend và thực thi kiểm thử thủ công end-to-end trực tiếp trên phần cứng thật để thu thập log vận hành.

1.5 Đóng góp chính của đồ án

Kết quả thực hiện đồ án mang lại một số đóng góp chính về mặt giải pháp kỹ thuật tự phát triển thay vì sử dụng các framework đóng gói sẵn của hãng. Bảng 1.1 tóm tắt các đóng góp và ý nghĩa thực tế tương ứng của chúng.

Bảng 1.1: Đóng góp chính và ý nghĩa kỹ thuật của đồ án

Giải pháp phát triển	Ý nghĩa thực tế và kỹ thuật
Kiến trúc Gateway C đơn tiến trình tích hợp trực tiếp MQTT client	Loại bỏ hoàn toàn tiến trình trung gian Python IPC cũ, giúp giảm thiểu độ trễ truyền dẫn lệnh xuống thiết bị và tiết kiệm tài nguyên CPU/RAM trên thiết bị host nhúng.
Mô hình theo dõi vòng đời lệnh điều khiển (Command Lifecycle)	Cho phép theo dõi tường minh trạng thái xử lý gói tin qua các pha (accepted, queued, sent, executed, failed, timeout), giải quyết bài toán lệnh bị treo hoặc mất gói vô tuyến mà cloud không phát hiện được.
Chuẩn hóa mô hình nhận dạng thiết bị qua device_id logic	Tách biệt định danh cơ sở dữ liệu khỏi địa chỉ phần cứng cố định (EUI-64) hoặc địa chỉ động (16-bit nwk_addr), tăng tính bảo mật và cho phép linh hoạt thay thế phần cứng lỗi.
Bộ kịch bản kiểm thử E2E tích hợp evidence thực tế	Cung cấp phương pháp luận kiểm thử rõ ràng từ mức đơn vị đến tích hợp hệ thống, đi kèm các bằng chứng vận hành (log trace) chứng minh hệ thống hoạt động chính xác.

Các đóng góp trên được minh chứng thông qua kết quả kiểm thử thực tế tại Chương 5, cho thấy tính khả thi và độ tin cậy của giải pháp tự phát triển trong môi trường nghiên cứu phòng thí nghiệm.

1.6 Cấu trúc báo cáo

Nội dung báo cáo đồ án tốt nghiệp được tổ chức thành 6 chương chính sau đây:

Chương 1 – Tổng quan đề tài: Giới thiệu bối cảnh, lý do chọn đề tài, xác định mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Cơ sở lý thuyết và công nghệ nền: Trình bày nền tảng lý thuyết về giao thức mạng Zigbee, thư viện cluster ZCL, giao thức MQTT, giao tiếp nối tiếp UART/ASH và các kit vi điều khiển phần cứng được sử dụng.

Chương 3 – Phân tích yêu cầu và thiết kế hệ thống: Phân tích các yêu cầu chức năng, phi chức năng, xây dựng sơ đồ kiến trúc tổng thể, mô hình dữ liệu, đặc tả giao diện API và hợp đồng trao đổi bản tin MQTT Contract.

Chương 4 – Triển khai hệ thống: Trình bày chi tiết quá trình lập trình firmware thiết bị, viết ứng dụng Gateway C, lập trình Cloud Backend bằng FastAPI và giải quyết các lỗi thực tế khi deploy lên EC2.

Chương 5 – Kiểm thử và đánh giá: Thiết lập kế hoạch kiểm thử, trình bày kết quả chạy các test case cụ thể, ghi nhận các bằng chứng logs/traces và phân tích ưu điểm cũng như hạn chế của hệ thống.

Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, tự đánh giá năng lực tích lũy của bản thân và đề xuất các hướng nghiên cứu mở rộng trong tương lai như bảo mật TLS, cập nhật OTA và tích hợp WebSocket.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN

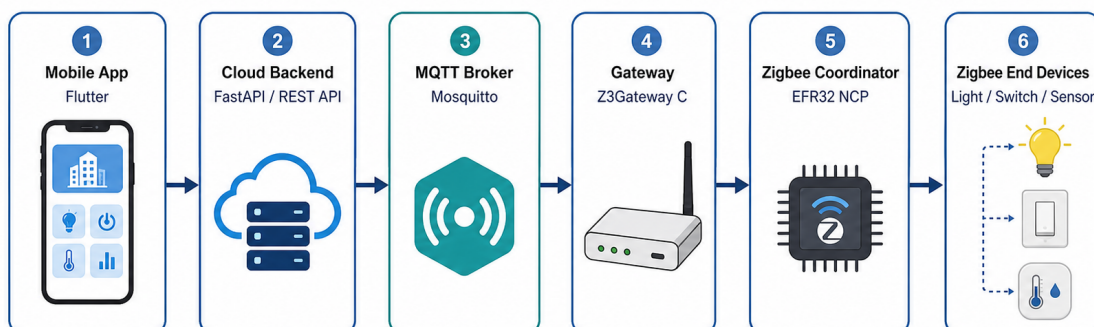
Chương 2 trình bày các cơ sở lý thuyết và công nghệ được sử dụng để xây dựng hệ thống IoT Zigbee Smart Building. Nội dung gồm mô hình IoT trong tòa nhà thông minh, mạng Zigbee, mô hình thiết bị theo Zigbee Cluster Library, vai trò của Gateway, giao thức MQTT, giao tiếp giữa Host và EFR32 NCP, Cloud Backend, Mobile App, firmware/OTA và một số vấn đề bảo mật cơ bản. Các phần dưới đây được trình bày ở mức nền tảng để làm cơ sở cho phần thiết kế và triển khai ở các chương sau, không đi vào log chạy thử hay kết quả thực nghiệm.

2.1 Tổng quan IoT và Smart Building

Internet of Things (IoT), hay Internet vạn vật, là mô hình kết nối các thiết bị vật lý với phần mềm thông qua mạng truyền thông. Thiết bị trong hệ thống IoT thường có cảm biến, bộ xử lý, phần mềm nhúng và khả năng trao đổi dữ liệu với các thành phần khác. Trong phạm vi đồ án, IoT được xét theo hướng ứng dụng trong một phòng/lab thông minh, nơi thiết bị có thể gửi trạng thái, nhận lệnh điều khiển và phối hợp với phần mềm phía trên.

Smart Building là một hướng ứng dụng của IoT trong môi trường tòa nhà. Hệ thống thường bao gồm sensor (cảm biến), actuator (thiết bị chấp hành), Gateway (cổng kết nối), Cloud Backend (dịch vụ phía máy chủ) và App (ứng dụng người dùng). Sensor dùng để thu thập trạng thái, ví dụ cảm biến hiện diện phát hiện có người trong phòng. Actuator thực hiện hành động vật lý như bật/tắt đèn hoặc thay đổi trạng thái thiết bị. Gateway nối mạng thiết bị cục bộ với hệ thống phần mềm. Cloud Backend lưu dữ liệu, quản lý thiết bị và cung cấp Application Programming Interface (API) cho App.

Một luồng IoT cơ bản thường bắt đầu từ việc thu thập dữ liệu, truyền dữ liệu, xử lý dữ liệu, phát sinh lệnh điều khiển và hiển thị lại trạng thái cho người dùng. Trong đồ án này, người dùng thao tác trên Flutter Mobile App; App gọi FastAPI Cloud REST API; Cloud trao đổi trạng thái và lệnh qua Mosquitto MQTT Broker; Z3Gateway C nhận bản tin MQTT và chuyển thành Zigbee command qua EFR32 NCP; thiết bị Zigbee phản hồi trạng thái theo chiều ngược lại.



Hình 2.1: Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building

Cách tổ chức trên giúp tách rõ trách nhiệm giữa các lớp. Mạng Zigbee xử lý truyền thông cục bộ cho thiết bị công suất thấp. Gateway đảm nhiệm chuyển đổi giữa Zigbee và MQTT. Cloud lưu dữ liệu và cung cấp API. App tập trung vào giao diện giám sát và điều khiển. Khi thiết kế hệ thống Smart Building, nhóm thực hiện cân cân bằng giữa xử lý cục bộ và xử lý trên Cloud. Xử lý cục bộ giúp giảm độ trễ trong một số tình huống, còn Cloud thuận lợi hơn cho lưu trữ, quản lý từ xa và mở rộng dữ liệu.

2.2 Zigbee và IEEE 802.15.4

Zigbee là giao thức truyền thông không dây công suất thấp, thường được dùng trong các hệ thống cảm biến và điều khiển [1]. Đặc điểm của Zigbee là tốc độ dữ liệu không cao nhưng tiêu thụ năng lượng thấp, hỗ trợ nhiều thiết bị và có khả năng tổ chức mạng linh hoạt. Các đặc điểm này phù hợp với Smart Building, vì các thiết bị như đèn, công tắc và cảm biến hiện diện chủ yếu trao đổi trạng thái hoặc lệnh ngắn, không truyền dữ liệu dung lượng lớn.

IEEE 802.15.4 là chuẩn ở tầng vật lý và tầng Medium Access Control (MAC) cho mạng cá nhân không dây công suất thấp. Zigbee sử dụng nền tảng này và bổ sung thêm các tầng phía trên như network layer, security và application layer [3]. Có thể hiểu IEEE 802.15.4 quy định cách truyền nhận khung dữ liệu ở mức radio, còn Zigbee quy định thêm cách thiết bị tạo mạng, tham gia mạng, định tuyến và biểu diễn chức năng ứng dụng.

Trong một hệ thống IoT, Zigbee không thay thế hoàn toàn Wi-Fi hoặc Ethernet. Zigbee phù hợp hơn ở lớp thiết bị cục bộ, nơi yêu cầu tiết kiệm năng lượng và truyền các bản tin nhỏ. Đối lại, thiết bị Zigbee thường không giao tiếp trực tiếp với Internet, nên hệ thống cần Gateway làm cầu nối lên Cloud hoặc App.

Bảng 2.1: Đặc điểm chính của Zigbee trong hệ thống Smart Building

Đặc điểm	Ý nghĩa trong hệ thống
Công suất thấp	Phù hợp với cảm biến, công tắc và thiết bị nhúng cần hoạt động lâu dài.
Băng thông thấp	Không phù hợp truyền dữ liệu lớn, nhưng đủ cho trạng thái, sự kiện và lệnh điều khiển.
Hỗ trợ mesh	Có thể mở rộng vùng phủ thông qua các thiết bị có khả năng định tuyến.
Mô hình ZCL	Chuẩn hóa chức năng thiết bị thành cluster, attribute và command.
Cần Gateway	Thiết bị Zigbee không giao tiếp trực tiếp với App/Cloud qua Internet, nên cần Gateway làm cầu nối.

Trong đồ án, Zigbee được dùng cho lớp thiết bị cục bộ. EFR32 NCP đóng vai trò radio coordinator, còn các thiết bị như Light, Switch và Occupancy Sensor là các thiết bị Zigbee trong mạng. Nhờ đó, phần thiết bị nhúng được tách khỏi phần Cloud/App, đồng thời vẫn có thể đồng bộ trạng thái và nhận lệnh điều khiển thông qua Gateway.

2.3 Vai trò thiết bị trong mạng Zigbee

Mạng Zigbee phân chia thiết bị theo vai trò để mạng có thể hình thành, duy trì kết nối và truyền dữ liệu. Ba vai trò cơ bản gồm Coordinator, Router và End Device. Coordinator tạo và

quản lý mạng. Router có thể chuyển tiếp gói tin để mở rộng vùng phủ. End Device là thiết bị cuối, thực hiện chức năng ứng dụng như đo trạng thái hoặc nhận lệnh điều khiển.

Coordinator là thiết bị khởi tạo mạng Zigbee. Thiết bị này chọn kênh, thiết lập Personal Area Network (PAN), quản lý quá trình cho phép thiết bị khác tham gia mạng và thường liên quan đến Trust Center trong cơ chế bảo mật. Trong kiến trúc Host-NCP, vai trò coordinator radio có thể đặt trên EFR32 NCP, còn ứng dụng Gateway chạy ở phía Host.

Router là thiết bị có khả năng chuyển tiếp gói tin cho các node khác. Trong môi trường tòa nhà, Router giúp mở rộng vùng phủ khi thiết bị không thể liên lạc trực tiếp với Coordinator. Tuy nhiên, Router thường cần nguồn cấp ổn định hơn so với End Device vì phải duy trì chức năng định tuyến.

End Device là thiết bị cuối trong mạng. Một End Device có thể là cảm biến hiện diện, công tắc hoặc đèn. Thiết bị này tập trung vào chức năng ứng dụng, ví dụ gửi trạng thái có người/không có người hoặc nhận lệnh bật/tắt. Với các thiết bị dùng pin, End Device có thể được tối ưu để tiết kiệm năng lượng bằng cách ngủ trong một số khoảng thời gian.

Bảng 2.2: Vai trò thiết bị trong mạng Zigbee

Vai trò	Nhiệm vụ chính	Ví dụ trong đề tài
Coordinator	Tạo mạng, quản lý mạng và hỗ trợ thiết bị tham gia mạng.	EFR32 NCP gắn với Gateway.
Router	Chuyển tiếp gói tin, mở rộng vùng phủ mạng.	Thiết bị luôn cấp nguồn nếu được cấu hình hỗ trợ routing.
End Device	Thực hiện chức năng ứng dụng, gửi trạng thái hoặc nhận lệnh.	Light, Switch, Occupancy Sensor.

Phần này không đi sâu vào thuật toán định tuyến của Zigbee. Mục tiêu là làm rõ vì sao Gateway cần một coordinator radio để quản lý mạng, và vì sao các thiết bị như light, switch, sensor có thể được tổ chức như các node trong cùng một mạng Zigbee.

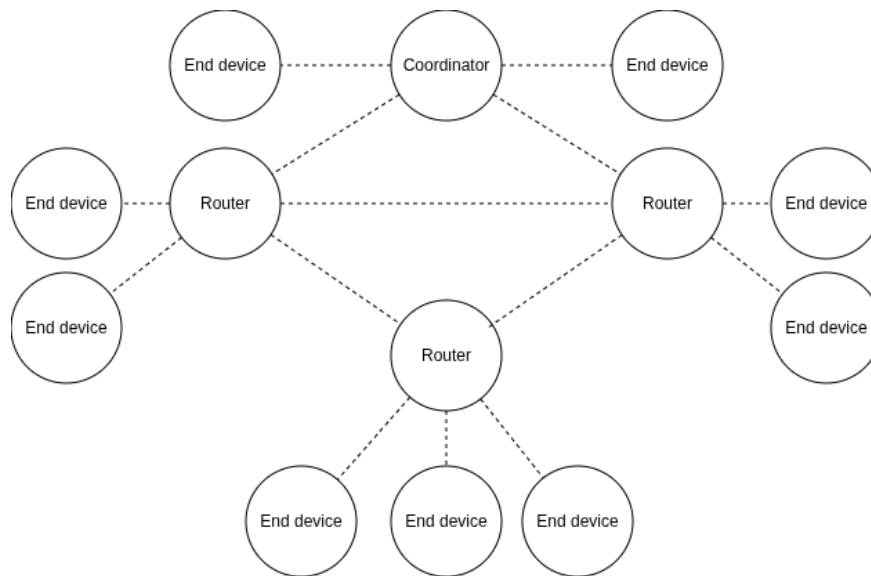
2.4 Topology mạng Zigbee

Topology mô tả cách các node trong mạng được tổ chức và liên kết với nhau. Trong Zigbee, ba dạng topology thường gặp là star, tree và mesh. Việc lựa chọn topology ảnh hưởng trực tiếp đến vùng phủ, độ phức tạp khi quản lý mạng và khả năng mở rộng của hệ thống.

Với star topology, các thiết bị trao đổi dữ liệu thông qua một node trung tâm. Mô hình này đơn giản, dễ kiểm soát và phù hợp với phạm vi nhỏ. Hạn chế của nó là vùng phủ phụ thuộc nhiều vào node trung tâm; nếu thiết bị ở xa hoặc bị vật cản, chất lượng kết nối có thể giảm.

Tree topology tổ chức mạng theo dạng phân cấp. Cách tổ chức này phù hợp khi muốn quản lý theo nhánh, ví dụ theo tầng hoặc theo khu vực trong tòa nhà. Hạn chế là các node phía dưới phụ thuộc vào node trung gian trong nhánh; khi node trung gian gặp sự cố, các node phía sau có thể bị ảnh hưởng.

Mesh topology cho phép một số node chuyển tiếp gói tin qua nhiều đường khác nhau. Đây là topology đáng chú ý trong Smart Building vì môi trường tòa nhà thường có tường, vật cản và khoảng cách giữa các thiết bị. Mesh giúp mở rộng vùng phủ tốt hơn star topology, nhưng việc quản lý mạng và debug cũng phức tạp hơn.



Hình 2.2: Minh họa topology dạng mesh trong mạng Zigbee

Trong đồ án, Zigbee được lựa chọn để tạo mạng cục bộ cho các thiết bị trong phòng/lab. Dù số lượng thiết bị ở giai đoạn đồ án chưa lớn, nền tảng mesh của Zigbee vẫn có ý nghĩa vì nó cho phép hệ thống mở rộng sang nhiều node hơn khi cần.

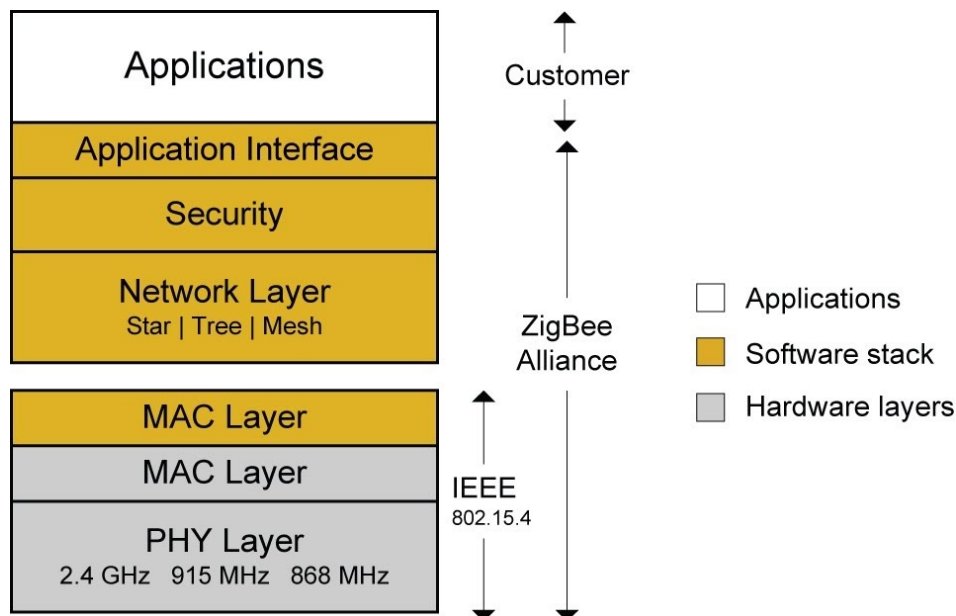
2.5 Zigbee Protocol Stack

Protocol Stack là tập hợp các tầng giao thức cùng tham gia vào quá trình truyền thông. Mỗi tầng đảm nhiệm một nhóm chức năng riêng, từ truyền tín hiệu vô tuyến ở tầng thấp đến biểu diễn lệnh ứng dụng ở tầng cao. Cách phân tầng này giúp hệ thống mạng dễ chuẩn hóa hơn, đồng thời giúp người phát triển xác định lỗi đang nằm ở tầng truyền dẫn, tầng mạng hay tầng ứng dụng.

Tầng Physical (PHY) và MAC của Zigbee dựa trên IEEE 802.15.4. Tầng PHY liên quan đến tần số, kênh truyền và việc phát/thu tín hiệu vô tuyến. Tầng MAC quản lý cách các node truy cập môi trường truyền, định dạng khung dữ liệu cơ bản và một số cơ chế trao đổi ở mức liên kết.

Tầng Network xử lý địa chỉ mạng, quá trình hình thành mạng, quá trình tham gia mạng và định tuyến. Khi một thiết bị mới join mạng, tầng này hỗ trợ cấp địa chỉ và xác định cách thiết bị gửi dữ liệu đến node đích. Với topology mesh, tầng Network có vai trò đáng kể vì gói tin có thể đi qua node trung gian.

Ở phía trên, Application Support (APS) và Application/ZCL xử lý truyền thông ở mức ứng dụng. APS hỗ trợ addressing và binding, còn ZCL định nghĩa cluster, attribute và command. Nhờ đó, lệnh bật/tắt đèn trong hệ thống Zigbee được biểu diễn theo mô hình chuẩn, thay vì dùng chuỗi dữ liệu tự do do từng nhóm tự đặt.



Hình 2.3: Kiến trúc tầng giao thức Zigbee

Ưu điểm của Protocol Stack chuẩn là tính tương thích và khả năng mở rộng tốt hơn. Hạn chế là người phát triển phải nắm thêm nhiều tầng giao thức khi debug. Với đồ án này, lợi ích của việc dùng stack chuẩn lớn hơn độ phức tạp phát sinh, vì hệ thống cần làm việc với nhiều loại thiết bị và cần ánh xạ rõ chức năng của từng thiết bị.

2.6 Zigbee Cluster Library và mô hình thiết bị

Zigbee Cluster Library (ZCL) là bộ đặc tả chuẩn hóa cách mô tả chức năng của thiết bị Zigbee [2]. Nếu các tầng thấp của Zigbee quy định cách dữ liệu đi qua mạng, ZCL quy định dữ liệu đó mang ý nghĩa gì ở tầng ứng dụng. Đây là phần cần thiết trong đồ án vì hệ thống sử dụng các thiết bị như Light, Switch và Occupancy Sensor.

Trong ZCL, cluster là nhóm chức năng của thiết bị. Attribute là trạng thái hoặc thuộc tính nằm trong cluster. Command là lệnh được gửi đến thiết bị hoặc sự kiện do thiết bị phát ra. Ba khái niệm này tạo ra cách biểu diễn thống nhất giữa Gateway và thiết bị Zigbee. Ví dụ, lệnh bật/tắt đèn có thể được biểu diễn bằng On/Off Cluster thay vì một chuỗi text tự định nghĩa.

Bảng 2.3: Một số cluster liên quan đến đèn

Cluster	Identifier	Vai trò
On/Off Cluster	0x0006	Biểu diễn chức năng bật/tắt, phù hợp với light hoặc switch.
Level Control Cluster	0x0008	Biểu diễn mức độ, ví dụ độ sáng của đèn nếu thiết bị hỗ trợ.
Occupancy Sensing Cluster	0x0406	Biểu diễn trạng thái phát hiện hiện diện, phù hợp với occupancy sensor.

Trong luồng điều khiển, App tạo yêu cầu ở mức người dùng, Cloud chuyển yêu cầu thành command logic, Gateway ánh xạ command sang action Zigbee, Zigbee stack đóng gói action theo cluster/command và thiết bị thực hiện lệnh. Ở chiều ngược lại, thiết bị gửi trạng thái mới, Gateway nhận báo cáo và publish reported state để Cloud lưu trữ và App hiển thị.

Việc sử dụng ZCL khiến phần phát triển phức tạp hơn so với tự đặt một payload đơn giản. Tuy nhiên, ZCL giúp hệ thống tránh tình trạng mỗi loại thiết bị dùng một cách biểu diễn riêng. Với đồ án này, lợi ích đó thể hiện rõ ở việc Gateway có thể xử lý lệnh theo loại thiết bị và chức năng, ví dụ tách lệnh bật/tắt light khỏi trạng thái phát hiện hiện diện của sensor.

2.7 Gateway trong hệ thống IoT

Gateway là thành phần nối mạng thiết bị cục bộ với hệ thống phần mềm phía trên. Trong hệ thống IoT Zigbee, thiết bị Zigbee không trực tiếp giao tiếp với REST API hoặc App. Gateway vì vậy cần chuyển đổi giữa hai miền: một bên là mạng Zigbee với cluster, attribute và command; bên còn lại là Cloud, MQTT và các dữ liệu ứng dụng.

Trong đồ án này, Gateway được hiểu theo kiến trúc hiện tại: Z3Gateway C Host App kết nối với EFR32 NCP và tích hợp MQTT trực tiếp thông qua libmosquitto. Kiến trúc Python MQTT-to-IPC bridge cũ không được dùng làm mô hình chính cho báo cáo. Luồng đúng của hệ thống là Cloud/App trao đổi qua MQTT Broker; Z3Gateway C nhận command từ MQTT; Gateway gọi Zigbee action qua Ember AF/EZSP; thiết bị Zigbee phản hồi trạng thái; Gateway publish reported state hoặc event trở lại MQTT.

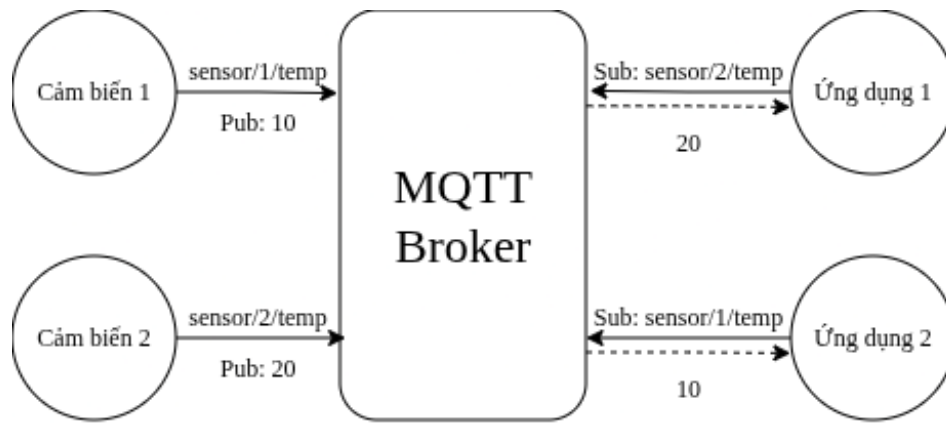
Gateway không chỉ chuyển tiếp dữ liệu thô. Thành phần này còn parse payload, xác định loại thiết bị, ánh xạ command theo device_type, gọi API của Zigbee stack, chuẩn hóa telemetry và publish trạng thái. Trong một số trường hợp, Gateway cũng có thể xử lý local automation ở mức cơ bản, ví dụ sự kiện từ switch hoặc sensor dẫn đến lệnh điều khiển light.

Khi đặt nhiều logic ở Gateway, hệ thống có thể phản hồi nhanh hơn và giảm phụ thuộc vào Cloud trong một số tình huống. Tuy nhiên, Gateway cũng trở thành thành phần khó bảo trì hơn nếu trách nhiệm bị mở rộng quá nhiều. Vì vậy, ranh giới hợp lý là Gateway xử lý kết nối thiết bị và các phản ứng cục bộ cần thiết, còn Cloud đảm nhiệm lưu trữ, API, lịch sử và quản lý người dùng.

2.8 MQTT và cơ chế Publish/Subscribe

Message Queuing Telemetry Transport (MQTT) là giao thức messaging nhẹ, được sử dụng phổ biến trong IoT để truyền telemetry và command giữa các thành phần phân tán [4, 5]. MQTT hoạt động theo mô hình publish/subscribe. Client không gửi dữ liệu trực tiếp cho nhau, mà publish message vào topic; broker nhận message và chuyển tiếp cho các client đang subscribe topic đó.

Các thuật ngữ cơ bản của MQTT gồm broker, client, topic, payload, publish, subscribe, Quality of Service (QoS), retained message và Last Will and Testament (LWT). Broker là trung tâm điều phối message. Client là thành phần kết nối tới broker, chẳng hạn Cloud Backend hoặc Gateway. Topic là đường dẫn logic dùng để phân loại message. Payload là nội dung message. Publish là hành động gửi message, còn subscribe là đăng ký nhận message.



Hình 2.4: Mô hình broker trung tâm trong MQTT

Trong hệ thống của đồ án, Cloud và Gateway cùng kết nối tới Mosquitto MQTT Broker. Ở chiều điều khiển, App gọi REST API lên Cloud, Cloud publish command request xuống topic mà Gateway subscribe. Gateway nhận message, parse payload và chuyển thành Zigbee action. Ở chiều trạng thái, Gateway nhận reported state hoặc event từ mạng Zigbee, sau đó publish lên MQTT để Cloud lưu vào database và App hiển thị lại cho người dùng.

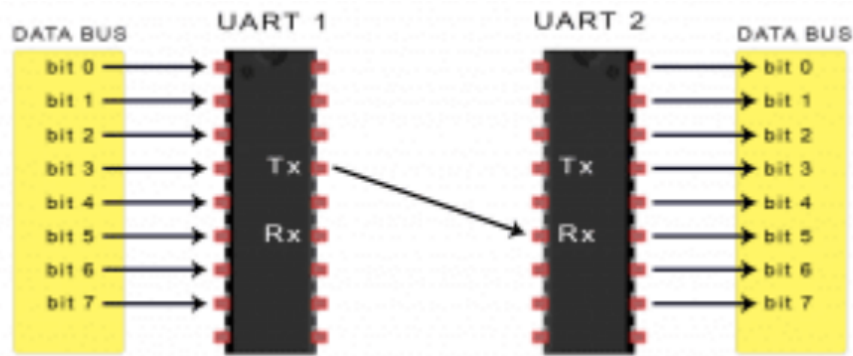
QoS quy định mức đảm bảo truyền message. QoS 0 gửi tối đa một lần, nhẹ nhất nhưng có thể mất message. QoS 1 đảm bảo message đến ít nhất một lần, nhưng bên nhận cần xử lý khả năng trùng lặp. QoS 2 đảm bảo đúng một lần ở mức giao thức, nhưng phức tạp và tốn tài nguyên hơn. Retained message cho phép broker giữ message cuối cùng của một topic, nhờ đó subscriber mới có thể nhận trạng thái gần nhất. LWT giúp broker phát message thay mặt client nếu client mất kết nối bất thường.

MQTT phù hợp với đồ án vì bản tin trạng thái và lệnh điều khiển thường nhỏ, gửi theo sự kiện và cần tách rời Cloud với Gateway. Hạn chế chính là hệ thống phải thiết kế topic và payload rõ ràng. Nếu contract MQTT không ổn định, việc debug và mở rộng hệ thống sẽ khó khăn hơn. Vì vậy, MQTT được xem là contract chính giữa Cloud và Gateway, còn chi tiết topic/payload cụ thể được trình bày ở phần thiết kế và triển khai.

2.9 UART, ASH và EZSP giữa Host và EFR32 NCP

Universal Asynchronous Receiver/Transmitter (UART) là chuẩn giao tiếp nối tiếp bất đồng bộ giữa hai thiết bị [7]. UART thường sử dụng các đường TX, RX và GND; hai bên truyền thông phải thống nhất baud rate, số bit dữ liệu, stop bit và parity. Trong đồ án, UART được xét chủ yếu như đường truyền vật lý/serial giữa Host Gateway và EFR32 NCP, không phải giao thức ứng dụng giữa Gateway và Cloud.

Network Co-Processor (NCP) là mô hình trong đó chip radio xử lý các phần Zigbee network/stack ở mức thấp hơn, còn Host chạy ứng dụng Gateway ở mức cao hơn. Host và NCP cần một cơ chế trao đổi lệnh, sự kiện và dữ liệu. Với nền tảng Silicon Labs, EmberZNet Serial Protocol (EZSP) và Asynchronous Serial Host (ASH) được dùng để hỗ trợ giao tiếp giữa Host và NCP [13, 14].



Hình 2.5: Minh họa giao tiếp UART giữa hai thành phần phần cứng

Điểm cần phân biệt là UART/EZSP/ASH thuộc biên giao tiếp giữa Host và NCP, còn application contract của hệ thống nằm ở MQTT và Zigbee/ZCL action mapping. Do đó, các frame UART tự định nghĩa trong tài liệu cũ không nên được xem là contract production hiện tại của đồ án.

Mô hình Host–NCP giúp nhóm thực hiện tận dụng stack Zigbee và phần radio của Silicon Labs, thay vì phải tự xử lý toàn bộ logic mạng ở mức thấp. Hạn chế là quá trình debug cần xem xét thêm lớp serial, EZSP/ASH và tương tác giữa Host với NCP. Phần này được trình bày ở mức vừa đủ để giải thích vì sao Gateway C có thể điều khiển mạng Zigbee thông qua EFR32 NCP.

2.10 REST API, Cloud Backend và Database

Cloud Backend là lớp phần mềm trung tâm ở phía server. Trong hệ thống Smart Building, Cloud Backend cung cấp REST API cho App, nhận trạng thái từ Gateway, lưu dữ liệu vào database và gửi command xuống Gateway khi người dùng hoặc rule automation yêu cầu. Nhờ có Backend, App không cần làm việc trực tiếp với Zigbee, cluster hoặc NCP.

Representational State Transfer (REST) là phong cách thiết kế API phổ biến trên nền HTTP. App có thể gọi API để lấy danh sách thiết bị, trạng thái hiện tại, lịch sử event hoặc gửi yêu cầu điều khiển. REST dễ triển khai và dễ kiểm thử, phù hợp với các thao tác quản lý thông thường. Hạn chế của REST là mô hình request/response không tối ưu cho cập nhật realtime liên tục nếu dùng một mình; với yêu cầu realtime cao hơn, hệ thống có thể cần thêm WebSocket hoặc cơ chế push.

Database lưu trữ các dữ liệu bền vững của hệ thống. Các nhóm dữ liệu thường gặp gồm device registry, device state, event history, command lifecycle và automation rule. Device registry mô tả thiết bị trong hệ thống. Device state biểu diễn trạng thái hiện tại hoặc lịch sử trạng thái. Event history hỗ trợ truy vết sự kiện. Command lifecycle cho biết lệnh đã được tạo, gửi, xác nhận hay thất bại. Automation rule mô tả điều kiện và hành động tự động.

Trong đồ án, FastAPI được sử dụng cho REST API, SQLAlchemy hỗ trợ Object-Relational Mapping (ORM), PostgreSQL dùng cho dữ liệu production và Paho MQTT giúp Cloud kết nối với broker. Ở Chương 2, nhóm thực hiện chỉ trình bày vai trò của các thành phần này ở mức công nghệ nền; danh sách endpoint, schema chi tiết và xử lý nghiệp vụ được đặt ở các chương sau.

Bảng 2.4: Vai trò của các thành phần Cloud trong hệ thống

Thành phần	Vai trò lý thuyết
REST API	Cung cấp giao diện HTTP để App lấy dữ liệu và gửi yêu cầu điều khiển.
MQTT Client	Subscribe trạng thái từ Gateway và publish command xuống Gateway.
Database	Lưu thiết bị, trạng thái, sự kiện, command và automation rule.
ORM	Ảnh xạ giữa object trong code backend và bảng dữ liệu trong database.

2.11 Mobile App và State Management

Mobile App là lớp tương tác trực tiếp với người dùng cuối. Trong hệ thống Smart Building, người dùng cần xem trạng thái thiết bị, gửi lệnh điều khiển và theo dõi các event hoặc automation rule. App vì vậy chuyển thao tác của người dùng thành request gửi lên Cloud Backend, đồng thời trình bày lại dữ liệu mà Backend cung cấp.

Flutter là framework UI cross-platform, cho phép xây dựng giao diện trên nhiều nền tảng từ cùng một codebase [17]. Trong đồ án, App gọi REST API qua HTTP. Cách tổ chức này giữ App ở lớp người dùng và Cloud, không để App giao tiếp trực tiếp với Zigbee hoặc Gateway. Nhờ đó, App tập trung vào giao diện và trải nghiệm sử dụng, còn Gateway và Cloud xử lý các phần truyền thông thiết bị.

State management là cách App quản lý và cập nhật trạng thái giao diện. Khi danh sách thiết bị, trạng thái đèn hoặc log automation thay đổi, giao diện cần được cập nhật nhất quán. Provider là một cơ chế state management phổ biến trong Flutter, giúp tách dữ liệu/trạng thái khỏi widget hiển thị. Với đồ án này, state management giúp App phản ánh đúng trạng thái thiết bị sau khi người dùng gửi lệnh hoặc khi Gateway báo trạng thái mới.

Ưu điểm của việc để App làm việc qua REST API là kiến trúc rõ ràng và dễ kiểm thử. Hạn chế là cập nhật realtime có thể chưa mượt nếu chỉ dựa vào request/response. Vì vậy, trong phạm vi Chương 2, App được trình bày như lớp monitoring/control cho người dùng; chi tiết màn hình và luồng thao tác được trình bày ở chương thiết kế hoặc triển khai.

2.12 Firmware, Bootloader và OTA

Firmware là phần mềm chạy trực tiếp trên thiết bị nhúng. Khác với ứng dụng mobile hoặc backend chạy trên hệ điều hành tổng quát, firmware làm việc gần hơn với phần cứng như radio, GPIO, cảm biến và bộ nhớ flash. Trong hệ thống Zigbee, firmware quyết định thiết bị hoạt động như light, switch, occupancy sensor hay NCP coordinator.

Bootloader là chương trình nhỏ chạy trước firmware chính, hỗ trợ nạp, kiểm tra hoặc cập nhật firmware [11]. Trong quá trình phát triển, firmware artifact như file .s37 có thể được dùng để flash thiết bị bằng công cụ của Silicon Labs. Artifact không phải là chức năng mà người dùng cuối thao tác trực tiếp, nhưng là đầu ra cần thiết khi build và nạp firmware cho board.

Over-the-Air (OTA) là cơ chế cập nhật firmware qua mạng thay vì flash thủ công qua máy tính [18]. Với Zigbee OTA, hệ thống thường cần OTA image, quản lý version, kiểm tra tính toàn vẹn, OTA server/client và quá trình truyền firmware theo từng phần. OTA có ý nghĩa khi số lượng thiết bị tăng lên hoặc thiết bị được lắp ở vị trí khó tiếp cận.

OTA cũng làm hệ thống phức tạp hơn vì liên quan đến phân phối firmware, xử lý lỗi cập

nhật, kiểm tra version, bảo mật image và khả năng khôi phục khi cập nhật thất bại. Do đó, trong báo cáo này, OTA được trình bày chủ yếu như nền tảng công nghệ và hướng phát triển. Nhóm thực hiện không khẳng định OTA end-to-end đã hoàn chỉnh nếu chưa có bằng chứng triển khai đầy đủ trong các chương sau.

2.13 Security trong Zigbee và IoT

Security trong hệ thống IoT cần được xem xét ở nhiều lớp, từ thiết bị, mạng Zigbee, Gateway, MQTT Broker, Cloud Backend, Database đến App. Với Smart Building, sự cố bảo mật có thể dẫn đến mất dữ liệu hoặc làm sai lệch hành vi điều khiển thiết bị vật lý. Vì vậy, báo cáo cần nêu các nguyên tắc bảo mật cơ bản trước khi đi vào phần thiết kế hệ thống.

Ở lớp Zigbee, các khái niệm cần chú ý gồm network key, application security và Trust Center. Network key giúp bảo vệ truyền thông trong cùng một mạng Zigbee. Trust Center liên quan đến việc quản lý thiết bị tham gia mạng và phân phối khóa trong một số mô hình bảo mật. Khi thiết bị join mạng, hệ thống cần kiểm soát thiết bị nào được phép tham gia và quá trình trao đổi khóa được thực hiện như thế nào.

Ở lớp MQTT và Cloud, broker cần có authentication, phân quyền topic và trong môi trường production nên cân nhắc Transport Layer Security (TLS). Cloud Backend cần kiểm soát quyền truy cập API, kiểm tra dữ liệu đầu vào và bảo vệ database. App cũng cần xử lý token hoặc thông tin đăng nhập cẩn thận để tránh người dùng không hợp lệ gửi command điều khiển thiết bị.

Firmware security là lớp bảo mật quan trọng khác. Firmware signing giúp giảm rủi ro thiết bị nhận firmware giả mạo. Với OTA, yêu cầu bảo mật còn cao hơn vì firmware được truyền qua mạng. Tuy nhiên, các cơ chế như install-code Trust Center join, APS encryption hardening, GBL signing hoặc MQTT mTLS chỉ được xem là nền tảng hoặc hướng phát triển nếu chưa có bằng chứng hoàn thiện trong hệ thống hiện tại.

Việc tăng cường security giúp hệ thống an toàn hơn nhưng cũng làm quá trình triển khai, vận hành và debug phức tạp hơn. Vì vậy, đồ án cần trình bày đúng phạm vi: nêu nguyên tắc bảo mật nền tảng, chỉ khẳng định những phần có evidence, và đưa các cơ chế nâng cao vào hướng phát triển khi chưa triển khai đầy đủ.

2.14 Công nghệ sử dụng trong đồ án

Các công nghệ trong đồ án được tổ chức theo kiến trúc nhiều lớp. Thiết bị Zigbee nằm ở tầng nhúng và mạng cục bộ. Gateway chịu trách nhiệm chuyển đổi giữa Zigbee và MQTT. MQTT Broker làm trung gian messaging. Cloud Backend quản lý dữ liệu và API. Mobile App là giao diện cho người dùng cuối. Cách chia lớp này giúp hệ thống dễ giải thích, dễ kiểm thử theo từng phần và thuận lợi hơn khi mở rộng.

Bảng 2.5: Tóm tắt công nghệ nền được sử dụng trong đồ án

Lớp	Công nghệ	Vai trò trong hệ thống
Device	EFR32, EmberZNet/Gecko SDK, Zigbee stack	Chạy firmware cho light, switch, occupancy sensor hoặc NCP.
Gateway	Z3GatewayHost, C, libmosquitto	Kết nối Zigbee local network với MQTT/Cloud.
Broker	Eclipse Mosquitto	Trung gian publish/subscribe giữa Cloud và Gateway.
Backend	FastAPI, SQLAlchemy, Paho MQTT	Cung cấp REST API, xử lý MQTT và quản lý dữ liệu/lệnh.
Database	PostgreSQL, SQLite trong một số ngữ cảnh test	Lưu thiết bị, trạng thái, sự kiện, command và automation.
App	Flutter, Dart, Provider, HTTP	Giao diện monitoring/control cho người dùng cuối.

Tóm lại, Chương 2 cung cấp phần cơ sở để người đọc hiểu các lựa chọn kỹ thuật chính của đồ án: Zigbee cho mạng thiết bị, Z3Gateway C cho lớp kết nối với coordinator radio, MQTT cho trao đổi trạng thái/lệnh, REST API và database cho lớp Cloud, Flutter cho giao diện người dùng. Các khái niệm này là tiền đề cho phân phân tích yêu cầu, thiết kế kiến trúc, triển khai và đánh giá ở các chương tiếp theo.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Sau phần cơ sở lý thuyết, chương này chuyển sang mô tả hệ thống trong phạm vi đồ án. Hiện tại, hệ thống được triển khai theo tuyến điều khiển Cloud-to-Gateway. Ứng dụng di động Flutter gửi yêu cầu qua REST API tới Cloud Backend FastAPI. Cloud lưu dữ liệu bằng PostgreSQL, phát lệnh qua MQTT Broker Mosquitto, sau đó Gateway Z3GatewayHost viết bằng C nhận lệnh bằng thư viện libmosquitto và chuyển xuống mạng Zigbee thông qua EFR32 NCP. Các thiết bị Zigbee cuối gồm đèn, công tắc và cảm biến chuyển động.

3.1 Phạm vi và giả định thiết kế

Phân thiết kế của đồ án tập trung vào một hệ thống Smart Building quy mô nhỏ. Hệ thống cần cho phép người dùng theo dõi trạng thái thiết bị, điều khiển đèn, xem sự kiện và cấu hình một phần automation rule. Các chức năng được mô tả trong chương này bám theo những thành phần đã có trong mã nguồn, không mở rộng sang các phần chưa có bằng chứng triển khai.

Một số khái niệm cơ bản:

- **Contract (hợp đồng dữ liệu):** quy ước giữa các module, ví dụ topic MQTT, cấu trúc JSON, REST request và response.
- **Reported state (trạng thái báo cáo):** trạng thái do thiết bị hoặc Gateway gửi lên Cloud, ví dụ đèn đang bật, mức sáng hiện tại, thiết bị còn reachable hay không.
- **Command lifecycle (vòng đời lệnh):** các bước một lệnh đi qua từ lúc Mobile tạo lệnh, Cloud lưu lệnh, Gateway nhận lệnh, gửi xuống Zigbee và trả kết quả.

Bảng 3.1 tóm tắt phạm vi được dùng khi viết chương này.

Bảng 3.1: Phạm vi thiết kế trong đồ án

Nhóm chức năng	Được mô tả trong chương	Không khẳng định trong phạm vi hiện tại
Điều khiển thiết bị	Điều khiển đèn qua Mobile, REST API, MQTT và Gateway Zigbee.	Không xem công tắc và cảm biến chuyển động là thiết bị có thể nhận lệnh trực tiếp.
Theo dõi trạng thái	Lưu và hiển thị reported state, registry, event và gateway health.	Không khẳng định mọi loại telemetry đều đã hoàn chỉnh cho sản phẩm thương mại.
Automation	Cloud có CRUD automation rule và publish desired automation. Gateway có rule engine local cho switch-to-light khi bật cấu hình môi trường.	Không khẳng định Cloud-created automation đã được Gateway tự động áp dụng end-to-end.
Bảo mật	MQTT Broker có username/password và ACL.	Không khẳng định đã có mTLS, multi-user auth hoàn chỉnh hoặc install-code commissioning.

3.2 Phân tích yêu cầu hệ thống

3.2.1 Tác nhân sử dụng hệ thống

Hệ thống có ba nhóm tác nhân chính. Người dùng thao tác qua Mobile App. Cloud Backend tiếp nhận request, lưu dữ liệu và làm cầu nối với MQTT Broker. Gateway là tác nhân kỹ thuật nằm gần mạng Zigbee, chịu trách nhiệm nhận lệnh MQTT và phát lệnh xuống thiết bị.

Bảng 3.2: Các tác nhân chính trong hệ thống

Tác nhân	Vai trò	Giao tiếp chính
Người dùng Mobile	Xem dashboard, xem danh sách thiết bị, điều khiển đèn, xem event log và thao tác automation rule.	REST API tới Cloud Backend.
Cloud Backend	Quản lý thiết bị, trạng thái, lệnh, sự kiện và automation. Thành phần này cũng publish/subscribe MQTT để trao đổi với Gateway.	REST, PostgreSQL, MQTT.
Gateway Z3GatewayHost	Nhận MQTT command, đưa lệnh vào queue, xử lý trong vòng lặp chính và gọi Zigbee stack để điều khiển thiết bị.	MQTT qua libmosquitto, Zigbee qua EFR32 NCP.
Thiết bị Zigbee	Đèn báo cáo trạng thái và nhận lệnh On/Off. Công tắc phát sự kiện toggle. Cảm biến chuyển động báo occupancy.	Zigbee Cluster Library.

3.2.2 Yêu cầu chức năng

Các yêu cầu chức năng được rút ra từ đường chạy hiện tại của hệ thống. Bảng 3.3 dùng mã FR để thuận tiện đối chiếu khi thiết kế API, cơ sở dữ liệu và MQTT contract.

Bảng 3.3: Các yêu cầu chức năng chính

Mã	Yêu cầu	Ghi chú thiết kế
FR-01	Người dùng xem được danh sách thiết bị, thông tin phòng và trạng thái mới nhất của từng thiết bị.	Dữ liệu lấy từ Cloud qua REST API.
FR-02	Người dùng điều khiển bật/tắt đèn từ Mobile App.	Lệnh đi qua Cloud, MQTT Broker, Gateway rồi tới mạng Zigbee.
FR-03	Gateway gửi reported state, registry, event và health lên Cloud.	Các bản tin uplink đi qua MQTT.
FR-04	Cloud lưu command, state, event và automation rule để Mobile truy vấn lại.	PostgreSQL là nơi lưu trữ chính.
FR-05	Người dùng xem được event log và trạng thái xử lý command.	Mobile có cơ chế polling command sau khi gửi lệnh.
FR-06	Người dùng tạo, sửa, bật/tắt và xóa automation rule ở phía Cloud.	Rule được lưu ở Cloud và publish lên desired automation topic; Gateway hiện chưa tự động consume topic này.
FR-07	Hệ thống hỗ trợ mở/đóng commissioning và rediscover thiết bị qua Gateway API.	Đây là lệnh Gateway-level, không phải lệnh điều khiển trực tiếp một thiết bị cụ thể.

3.2.3 Yêu cầu phi chức năng

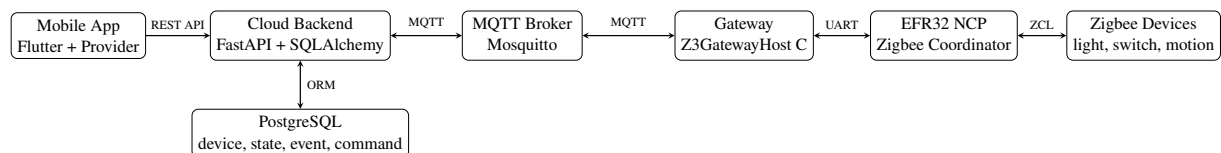
Yêu cầu phi chức năng trong đồ án được đặt ở mức phù hợp với hệ thống thử nghiệm. Các tiêu chí này dùng để định hướng thiết kế, không được hiểu là số đo đã kiểm chứng trên môi trường sản xuất.

Bảng 3.4: Các yêu cầu phi chức năng

Tiêu chí	Yêu cầu thiết kế	Cách đáp ứng trong hệ thống
Độ trễ thao tác	Lệnh bật/tắt đèn cần đi qua ít tầng trung gian nhất có thể trong mô hình Cloud-to-Gateway.	Mobile gửi REST command, Cloud publish MQTT, Gateway xử lý queue và gọi Zigbee stack.
Tính nhất quán dữ liệu	Trạng thái hiển thị trên Mobile phải dựa trên reported state từ Gateway thay vì chỉ dựa vào thao tác người dùng.	Gateway publish reported state retained; Cloud lưu device state.
Khả năng quan sát	Người vận hành cần biết Gateway online, health và command reply.	Gateway publish online, health, event và command reply qua MQTT.
An toàn truy cập	MQTT Broker cần tách quyền publish/subscribe theo cấu hình broker.	Mosquitto dùng username/password và ACL.
Dễ bảo trì	Các tầng Mobile, Cloud, Broker, Gateway và firmware nên tách trách nhiệm rõ ràng.	Cloud dùng FastAPI router; Gateway tách MQTT, parser, dispatcher, light/switch logic và rule engine.

3.3 Kiến trúc tổng thể hệ thống

Kiến trúc runtime hiện tại gồm năm lớp: Mobile App, Cloud Backend, MQTT Broker, Gateway và mạng Zigbee. Docker compose production chứa các service Cloud API, Mosquitto và PostgreSQL. Mobile App và Gateway không nằm trong compose này; chúng chạy như thành phần riêng và kết nối vào Cloud/Broker qua network.



Hình 3.1: Kiến trúc runtime của hệ thống Smart Building trong đồ án

Trong kiến trúc này, Cloud không điều khiển trực tiếp thiết bị Zigbee. Cloud tạo command và phát bản tin MQTT. Gateway mới là nơi hiểu phần Zigbee runtime, ví dụ mapping command sang cluster On/Off, đẩy lệnh xuống EFR32 NCP và gửi command reply ngược lại Cloud.

Bảng 3.5: Trách nhiệm của các khối chính

Khối	Trách nhiệm chính	Dữ liệu ra/vào
Mobile App	Giao diện thao tác thiết bị, event, automation và settings.	REST request/response.
Cloud Backend	Xử lý REST API, lưu dữ liệu, publish command, subscribe reported state, event, registry và health.	JSON qua REST, MQTT envelope, bản ghi PostgreSQL.
MQTT Broker	Trung gian truyền bản tin giữa Cloud và Gateway.	Topic prefix: sb/v1/{tenant}/{site}/{gateway}.
Gateway Host	Nhận command, parse contract, route lệnh tới module phù hợp, publish reply và telemetry.	MQTT, command queue, Zigbee API.
Zigbee Devices	Thực hiện lệnh và báo cáo trạng thái/sự kiện.	ZCL command, ZCL report.

3.4 Thiết kế thiết bị và trạng thái

3.4.1 Device model

Device model của hệ thống hiện tại dùng các loại thiết bị light, switch, motion và unknown. Điểm dễ nhầm là firmware cảm biến có tên Occupancy Sensor, nhưng khi đi qua Gateway/Cloud/Mobile, loại thiết bị được chuẩn hóa là motion. Vì vậy trong chương này không dùng occupancy_sensor như một device type runtime.

Bảng 3.6: Ánh xạ loại thiết bị và chức năng runtime

Device type	Vai trò	State/Event chính	Command hỗ trợ
light	Thiết bị chấp hành, nhận lệnh bật/tắt.	power, level, reachable.	on, off end-to-end. set_level có trong schema nhưng Gateway hiện chưa xử lý hoàn chỉnh.
switch	Thiết bị phát sự kiện điều khiển cục bộ.	Event toggle; schema có thể lưu reachable, battery.	Không commandable trong Gateway hiện tại.
motion	Cảm biến chuyển động/occupancy.	occupancy với trạng thái occupied hoặc unoccupied khi có event phù hợp.	Không có lệnh điều khiển trực tiếp.
unknown	Thiết bị chưa phân loại được khi discovery.	Registry metadata nếu có.	Không dùng cho điều khiển.

3.4.2 State, event và command

Hệ thống tách ba loại dữ liệu để tránh lẫn giữa trạng thái hiện tại, sự kiện tức thời và yêu cầu điều khiển:

- **State:** ảnh chụp trạng thái gần nhất của thiết bị. Ví dụ đèn có power=on, level=100, reachable=true.
- **Event:** sự kiện xảy ra tại một thời điểm. Ví dụ công tắc phát toggle hoặc cảm biến báo occupancy_changed.

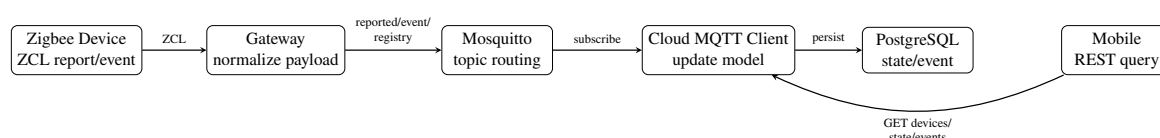
- **Command:** yêu cầu điều khiển do người dùng hoặc hệ thống tạo ra. Ví dụ Mobile yêu cầu bật đèn.

Cách tách này giúp Cloud không phải suy đoán trạng thái thiết bị chỉ từ thao tác trên Mobile. Sau khi gửi command, Mobile vẫn cần lấy lại command status hoặc chờ reported state mới để hiển thị đúng hơn.

3.5 Luồng dữ liệu chính

3.5.1 Luồng uplink: thiết bị báo cáo lên Cloud

Uplink là chiều dữ liệu từ mạng Zigbee đi lên Cloud. Đèn gửi ZCL report, công tắc gửi event, cảm biến motion gửi thay đổi occupancy. Gateway nhận các thông tin này, chuẩn hóa thành MQTT payload và publish lên Broker. Cloud subscribe các topic tương ứng, cập nhật database và cung cấp lại dữ liệu cho Mobile qua REST API.

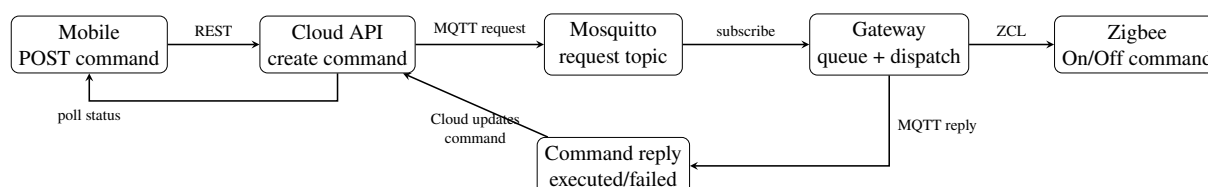


Hình 3.2: Luồng uplink đồng bộ trạng thái và sự kiện

Các bản tin reported state, registry và gateway health được thiết kế theo hướng giữ lại bản tin mới nhất trên Broker bằng retain flag ở phía Gateway. Với event và command reply, hệ thống không dùng retained message vì đây là dữ liệu theo thời điểm.

3.5.2 Luồng downlink: người dùng gửi lệnh điều khiển

Downlink là chiều điều khiển từ Mobile xuống thiết bị. Người dùng bấm bật/tắt đèn trên Mobile. Mobile gọi REST API tạo command. Cloud lưu command, chuyển command thành MQTT envelope và publish tới topic request của Gateway. Gateway nhận bản tin, đưa vào queue, xử lý trong main tick, gọi module light control rồi gửi lệnh Zigbee On/Off xuống thiết bị.



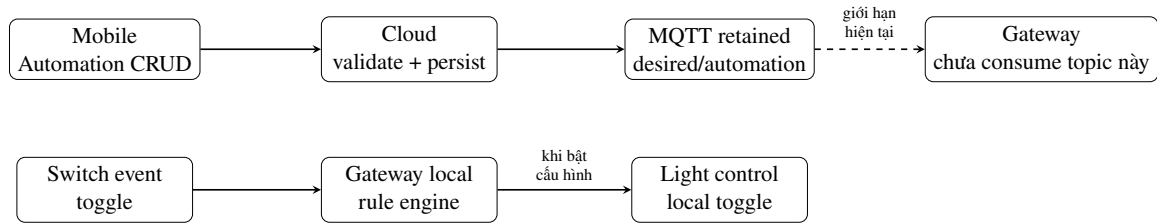
Hình 3.3: Luồng downlink và phản hồi command

Trong phần đánh giá, cần hiểu đúng ý nghĩa của trạng thái executed. Với light command hiện tại, trạng thái này phản ánh việc Gateway đã gửi lệnh xuống Zigbee ở mức truyền lệnh, chưa phải một xác nhận ứng dụng đầy đủ rằng thiết bị vật lý chắc chắn đã đổi trạng thái. Vì vậy reported state sau đó vẫn là nguồn đáng tin hơn để hiển thị trạng thái thiết bị.

3.5.3 Luồng automation

Automation hiện có hai phần cần phân biệt. Phía Cloud đã có API tạo/sửa/xóa automation rule, validate trigger/action và publish desired automation rule lên MQTT retained topic. Phía

Gateway có rule engine local cho tình huống switch-to-light, nhưng mặc định không có binding nếu chưa bật biến môi trường SB_RULES_SWITCH_TO_LIGHT=1. Trong mã nguồn hiện tại chưa thấy Gateway subscribe topic desired/automation.



Hình 3.4: Ranh giới thiết kế automation trong phiên bản hiện tại

Thiết kế này cho phép báo cáo rõ phần đã làm và phần còn mở. Cloud đã có nền tảng quản lý rule. Gateway đã có hướng xử lý local rule. Bước còn thiếu để có dynamic cloud-synced automation là cơ chế Gateway nhận desired automation, đồng bộ rule và chạy rule đó theo event thực tế.

3.6 Thiết kế MQTT contract

MQTT contract dùng prefix chung:

sb/v1/{tenant}/{site}/{gateway}

Trong môi trường thử nghiệm, các giá trị tenant, site và gateway có thể được cấu hình theo lab. Khi viết báo cáo, nên trình bày theo biến như trên để tránh khóa thiết kế vào một máy cụ thể.

Bảng 3.7: Các nhóm topic MQTT chính

Chiều	Topic	Payload chính	QoS/retain
Cloud tới Gateway	.../commands/{command_id}/request	Envelope có schema, msg_id, correlation_id và payload command.	Cloud publish QoS 0; Gateway subscribe QoS 1.
Gateway tới Cloud	.../commands/{command_id}/reply	command_id, device_id, status, reason.	QoS 1, không retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/reported	Reported state của thiết bị.	QoS 1, retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/event	Event như toggle hoặc occupancy_changed.	QoS 1, không retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/registry	Identity, endpoint, cluster và metadata.	QoS 1, retain.
Gateway tới Cloud	.../gateway/online .../gateway/health	Online/offline, uptime, mqtt_connected, known_device_count.	QoS 1, retain.
Cloud tới Broker	.../desired/automation/{automation_id}	Automation payload và cờ deleted.	QoS 0, retain.

Envelope command request được thiết kế để Gateway có đủ thông tin định danh và correlation khi trả kết quả. Ví dụ rút gọn:

Mã nguồn 3.1: Ví dụ MQTT command envelope

```
{
  "schema": "sb.v1.command",
  "msg_id": "msg-001",
  "ts": "2026-05-27T10:00:00Z",
  "tenant_id": "hust",
  "site_id": "lab01",
  "gateway_id": "gw-ubuntu-01",
  "source": "cloud-api",
  "correlation_id": "command-001",
  "payload": {
    "op": "device.command",
    "device_id": "light-001",
    "cluster": "0x0006",
    "command": "on"
  }
}
```

Reported state của đèn có dạng:

Mã nguồn 3.2: Ví dụ reported state của thiết bị đèn

```
{
  "device_id": "light-001",
  "device_type": "light",
  "eui64": "00124b...",
  "nwk_addr": "0x1234",
  "state": {
    "power": "on",
    "level": 100,
    "reachable": true
  }
}
```

3.7 Thiết kế Cloud Backend và cơ sở dữ liệu

3.7.1 REST API

Cloud Backend cung cấp REST API cho Mobile App. API được chia theo nhóm tài nguyên thay vì đưa toàn bộ logic vào một endpoint chung. Cách chia này phù hợp với FastAPI router và giúp Mobile chỉ gọi đúng nhóm dữ liệu cần dùng.

Bảng 3.8: Các nhóm REST API chính

Nhóm API	Endpoint tiêu biểu	Mục đích
Health	GET /health	Kiểm tra Cloud API đang chạy.
Devices	GET /api/devices, GET /api/devices/{id}, GET /api/devices/{id}/state	Lấy danh sách thiết bị và state mới nhất.
Commands	POST /api/devices/{id}/command GET /api/commands/{id}	Tạo lệnh điều khiển và truy vấn trạng thái lệnh.
Events	GET /api/events	Lọc và xem event theo thiết bị hoặc loại event.
Automations	/api/automations /api/automation-events	CRUD automation rule, bật/tắt rule và xem automation event.
Gateways	/api/gateways/{id}/commissioning/open /api/devices/{id}/rediscover	Gửi lệnh commissioning và rediscover tới Gateway.

Mobile App có login UI nhưng trong Cloud Backend hiện tại chưa có auth router tương ứng. Vì vậy, chương này không mô tả multi-user authentication như một chức năng đã hoàn thiện.

3.7.2 Cơ sở dữ liệu

Cloud Backend dùng ORM model làm nguồn thiết kế chính cho cơ sở dữ liệu. Khi service khởi động, hệ thống gọi cơ chế tạo bảng từ model. Các bảng chính được mô tả trong Bảng 3.9.

Bảng 3.9: Các bảng dữ liệu chính

Bảng	Vai trò	Liên quan tới
homes, rooms	Lưu cấu trúc không gian để gán thiết bị vào khu vực sử dụng.	Device list, dashboard.
users	Bảng người dùng đã có trong model.	Chưa có auth flow hoàn chỉnh trong Cloud API.
devices	Lưu định danh thiết bị, loại thiết bị, phòng và metadata.	Registry, REST devices.
device_states	Lưu state mới nhất hoặc state theo bản ghi tùy cách triển khai service.	Reported state, dashboard.
events	Lưu sự kiện từ Gateway, ví dụ switch toggle hoặc motion event.	Event log, automation input.
commands	Lưu command do Mobile/API tạo, trạng thái xử lý, timeout và thông tin target.	Downlink flow, command polling.
automations	Lưu automation rule, trigger, actions, version và trạng thái sync.	Automation CRUD.
automation_events	Lưu event liên quan đến automation, ví dụ sync hoặc execution event nếu Gateway gửi về.	Automation monitoring.

3.8 Thiết kế Gateway

Gateway hiện tại được thiết kế theo kiểu native C host application. Thành phần MQTT chạy với libmosquitto để nhận command và publish dữ liệu lên Broker. Sau khi nhận command request, Gateway không xử lý trực tiếp trong callback MQTT mà đưa vào queue. Vòng lặp chính rút command từ queue, parse envelope, kiểm tra target rồi chuyển tới module phù hợp.

Bảng 3.10: Các module logic chính trong Gateway

Module	Trách nhiệm
MQTT module	Kết nối Mosquitto, subscribe command request, publish reported state, registry, event, health và command reply.
Command parser/dispatcher	Chuẩn hóa payload sb.v1, xác định loại command và route tới gateway operation hoặc device operation.
Light control	Xử lý lệnh on/off cho đèn và gọi Zigbee On/Off cluster.
Switch logic	Nhận sự kiện từ công tắc; switch hiện không được thiết kế là thiết bị nhận command từ Cloud.
Telemetry receiver	Nhận ZCL attribute report/event từ Zigbee stack và chuyển thành MQTT payload.
Rule engine	Xử lý rule cục bộ switch-to-light khi cấu hình môi trường cho phép.

Điểm thiết kế đáng chú ý là Gateway không còn được mô tả như một Python MQTT-to-IPC bridge. Trong phiên bản hiện tại, đường chạy chính là Z3GatewayHost C dùng libmosquitto. Điều này giúp giảm một lớp trung gian, nhưng làm phần Gateway phụ thuộc nhiều hơn vào kỹ thuật tổ chức module trong C.

3.9 Thiết kế Mobile App

Mobile App được xây dựng bằng Flutter và dùng Provider cho quản lý state ở tầng giao diện. Ứng dụng gọi Cloud qua repository, trong đó các repository remote phụ trách gọi REST API thật. Các màn hình chính gồm Home/Dashboard, Automation, Provisioning và Settings.

Trong phạm vi đề án, Mobile App có các chức năng đáng chú ý sau:

- Xem danh sách thiết bị và trạng thái của thiết bị.
- Gửi lệnh bật/tắt đèn.
- Poll command status sau khi gửi lệnh để cập nhật kết quả thao tác.
- Xem event log.
- Tạo, sửa, bật/tắt và xóa automation rule qua Cloud API.
- Có màn hình provisioning ở mức placeholder, chưa xem là quy trình provisioning hoàn chỉnh.

Thiết kế này giữ Mobile ở vai trò client. Mobile không nói chuyện trực tiếp với Gateway hoặc Zigbee device. Mọi thao tác đi qua Cloud API để Cloud có thể lưu lịch sử command, event và trạng thái hệ thống.

3.10 Các quyết định thiết kế và trade-off

Trong quá trình thiết kế, nhóm chọn cách tách hệ thống thành các tầng rõ ràng thay vì để Mobile hoặc Cloud truy cập trực tiếp mạng Zigbee. Cách làm này phù hợp với bài toán Smart Building vì Gateway thường nằm trong mạng nội bộ, còn Mobile có thể truy cập Cloud từ bên ngoài.

Bảng 3.11: Các trade-off chính trong thiết kế

Quyết định	Lợi ích	Đánh đổi
Dùng MQTT giữa Cloud và Gateway	Phù hợp với mô hình publish/subscribe, dễ gửi uplink và downlink qua Broker.	Cần quản lý topic, QoS, retain và ACL cẩn thận.
Gateway native C thay vì bridge Python	Giảm lớp trung gian, gần với Zigbee host stack hơn.	Code C cần tổ chức module rõ để tránh khó bảo trì.
Lưu command/state/event ở Cloud	Mobile có thể truy vấn lại lịch sử và trạng thái.	Cloud trở thành điểm trung tâm cần database ổn định.
Retain reported state/registry/health	Subscriber mới có thể lấy trạng thái gần nhất nhanh hơn.	Không nên retain event hoặc command reply vì đó là dữ liệu theo thời điểm.
Automation tách Cloud CRUD và Gateway local rule	Có nền để phát triển dynamic automation sau này mà vẫn có rule local đơn giản.	Phiên bản hiện tại chưa phải cloud-synced automation end-to-end.

Từ các trade-off trên, hướng phát triển tiếp theo của hệ thống là hoàn thiện phần dynamic automation ở Gateway, bổ sung xác thực người dùng ở Cloud, tăng cường bảo mật MQTT và kiểm thử độ trễ thực tế trên nhiều thiết bị. Các phần này nên được trình bày là hướng mở rộng, không nên viết như chức năng đã hoàn thành trong phiên bản hiện tại.

CHƯƠNG 4

TRIỂN KHAI HỆ THỐNG

Chương này trình bày cách hiện thực các thành phần đã thiết kế ở Chương 3: firmware thiết bị Zigbee, Gateway native C, MQTT Broker, Cloud Backend, Mobile App và môi trường triển khai. Trọng tâm của chương là chỉ ra module, file cấu hình và luồng xử lý cụ thể trong mã nguồn, thay vì lặp lại phân lý thuyết hoặc đánh giá kết quả kiểm thử.

4.1 Môi trường triển khai và cấu trúc mã nguồn

Hệ thống được triển khai theo mô hình nhiều lớp. Ứng dụng Mobile App gọi Cloud Backend qua REST API; Cloud Backend lưu dữ liệu và trao đổi bản tin với Gateway thông qua Mosquitto MQTT Broker; Gateway native C chuyển đổi lệnh MQTT thành lệnh Zigbee gửi xuống EFR32 NCP; các thiết bị Zigbee phản hồi trạng thái ngược lại theo cùng đường truyền.

Bảng 4.1: Các thành phần triển khai chính trong mã nguồn

Thành phần	Công nghệ / vai trò	Evidence trong repo
Firmware Zigbee	EFR32, EmberZNet, NCP/Router/End Device	artifact/, end_devices/, docs/FIRMWARE_ARTIFACTS.md, docs/FLASHING.md
Gateway	Native C host app, lib-mosquitto, ASH/EZSP qua UART	gateway/Z3GatewayHost/app/app_mqtt.c, cmd_handler.c, device_dispatch.c, light_ctrl.c
Cloud Backend	FastAPI, SQLAlchemy async, PostgreSQL, MQTT client	cloud/app/main.py, models.py, mqtt_client.py, command_timeout.py, routers/
Mobile App	Flutter app gọi Cloud REST API	mobile_app/lib/data/, mobile_app/lib/presentation/, remote_device_repository.dart
Deployment	Docker Compose trên EC2 Ubuntu	deploy/docker-compose.prod.yml, deploy/deploy.ps1, deploy/ec2-setup.ps1

Các thông tin nhạy cảm như địa chỉ máy chủ thật, private key, mật khẩu MQTT hoặc mật khẩu database không được đưa trực tiếp vào báo cáo. Khi cần minh họa, chương này chỉ dùng tên biến môi trường hoặc placeholder.

4.2 Triển khai Firmware cho thiết bị Zigbee

Firmware cho các node Zigbee được xây dựng trên hệ sinh thái Silicon Labs, dùng Simplicity Studio và EmberZNet trong Gecko SDK. Repo hiện có artifact chắc chắn cho NCP, Z3Light và Z3Switch; riêng Occupancy/Motion có project/config nguồn nhưng chưa thấy artifact flash tương ứng trong thư mục artifact/, nên phần này chỉ mô tả ở mức triển khai project.

4.2.1 Triển khai Coordinator / NCP Board

Coordinator được triển khai theo mô hình Network Co-Processor (NCP). Firmware NCP chạy trên radio board EFR32 và giao tiếp với Gateway host qua UART bằng giao thức ASH/EZSP. Vì logic ứng dụng nằm ở Gateway, firmware NCP chủ yếu đảm nhiệm phần radio Zigbee: tạo mạng, giữ vai trò Trust Center và chuyển tiếp frame Zigbee theo yêu cầu từ host. Artifact flash của NCP được quản lý trong `artifact/ncp-uart-hw/manifest.json`.

4.2.2 Triển khai Router đèn (Z3Light Device)

Z3Light được cấu hình là Zigbee Router và expose các capability chính của thiết bị đèn, gồm On/Off và Level Control theo capability matrix của dự án. Khi nhận lệnh ZCL từ Gateway, firmware cập nhật trạng thái phần cứng của LED và phát Attribute Report về Coordinator. Artifact build của thiết bị này nằm trong `artifact/Z3Light/manifest.json`; đây là evidence chính để đưa Z3Light vào nhóm thiết bị đã có binary triển khai.

4.2.3 Triển khai Router công tắc (Z3Switch Device)

Z3Switch được dùng để phát sinh sự kiện điều khiển từ phía thiết bị. Trong triển khai hiện tại, switch có thể phát sự kiện toggle lên hệ thống và Gateway publish sự kiện này lên MQTT để Cloud/App quan sát. Một số luồng smoke test vẫn có thể dùng direct binding để switch điều khiển light ở lớp Zigbee, nhưng đây không phải là đường Cloud command chính. Artifact của Z3Switch được quản lý trong `artifact/Z3Switch/manifest.json`.

4.2.4 Triển khai cảm biến hiện diện (Occupancy Sensor)

Cảm biến hiện diện được mô hình hóa trong hệ thống bằng device type motion. Project nguồn có cấu hình Zigbee Cluster Library cho Occupancy Sensing Cluster tại `end_devices/Z3_Occupancy_Sensor/config/zcl/zcl_config.zap`. Vì chưa có artifact flash tương ứng trong `artifact/`, nội dung báo cáo chỉ khẳng định phần `project/config` đã chuẩn bị, còn việc kiểm chứng binary thực tế cần đặt ở Chương 5 nếu có log hoặc kết quả test.

4.3 Triển khai Gateway Service nhúng

Gateway được triển khai bằng native C trong thư mục `gateway/Z3GatewayHost/app/`. Thành phần này chạy trên Linux host, kết nối với NCP qua UART/ASH/EZSP và kết nối với MQTT Broker bằng libmosquitto. Cách tổ chức này thay thế lớp bridge Python cũ, nên đường điều khiển từ Cloud xuống thiết bị chỉ còn một tiến trình Gateway chịu trách nhiệm nhận MQTT, phân tích command và gọi lớp Zigbee host.

4.3.1 MQTT client và command queue

Module `app_mqtt.c` khởi tạo libmosquitto, kết nối Broker, subscribe topic `commands/+ /request` và publish các nhóm topic `gateway`, `reported state`, `device event` và `command reply`. Khi nhận command từ Cloud, module này đưa message vào hàng đợi nội bộ thay vì xử lý trực tiếp trong callback MQTT. Cách làm này giúp phần nhận mạng và phần điều khiển Zigbee tách nhau rõ hơn.

4.3.2 Command handler và điều khiển thiết bị

Sau khi message được lấy khỏi queue, Gateway chuyển payload sang `cmd_handler.c` và `sb_command.h` để parse command. Với lệnh điều khiển đèn, `device_dispatch.c` chọn module thiết bị phù hợp và gọi `light_ctrl.c` để đóng gói lệnh Zigbee Cluster Library (ZCL) gửi xuống NCP. Kết quả xử lý được publish ngược lại Cloud bằng topic reply.

Trong contract MQTT của dự án, `command_id` nằm trong topic `commands/{command_id}/request` và reply tương ứng nằm ở `commands/{command_id}/reply`. Trường `correlation_id` dùng để trace cùng một luồng nghiệp vụ, không thay thế khóa chính `command_id`. Vì vậy khi viết hoặc debug luồng command, Gateway và Cloud đều phải giữ nguyên cặp request/reply này.

4.3.3 Discovery, registry và rule cục bộ

Các module `device_discovery.c`, `device_registry.c` và `telemetry_rx.c` xử lý dữ liệu thiết bị từ mạng Zigbee, gồm join event, endpoint, cluster, EUI64 và trạng thái reported. Với automation tại biên, repo có `rule_engine.c` và các hook liên quan, nhưng phiên bản hiện tại không nên được mô tả như một rule engine động đã hoàn chỉnh. Cách viết an toàn là: Gateway đã có khung xử lý local rule và có thể hỗ trợ một số rule cố định, còn phần đồng bộ rule động giữa Cloud và Gateway vẫn là phạm vi mở rộng.

4.4 Triển khai Cloud Backend

Cloud Backend được xây dựng bằng Python, FastAPI, SQLAlchemy async và PostgreSQL. Điểm quan trọng trong triển khai là Cloud không điều khiển Zigbee trực tiếp. Cloud nhận yêu cầu từ Mobile App, lưu trạng thái vào database, publish command qua MQTT và chờ Gateway gửi reply hoặc reported state.

4.4.1 FastAPI app lifecycle và router

Tệp `cloud/app/main.py` khai báo FastAPI app, khởi tạo database, kết nối MQTT service trong lifespan và chạy timeout worker nền. Các router chính được tách theo nghiệp vụ:

- `routers/devices.py`: đọc danh sách thiết bị, chi tiết thiết bị và trạng thái hiện tại.
- `routers/commands.py`: tạo command điều khiển thiết bị và đọc trạng thái command.
- `routers/events.py`: truy vấn event history phục vụ quan sát hệ thống.
- `routers/automations.py`: quản lý rule automation ở phía Cloud.
- `routers/gateways.py` và `routers/provisioning.py`: mở/đóng commissioning, re-discover thiết bị và quản lý provisioning session.

4.4.2 Database model và command lifecycle

Các bảng chính được định nghĩa trong `cloud/app/models.py`, gồm Device, DeviceState, Event, Command, Automation và ProvisioningSession. Với command điều khiển, Cloud lưu bản ghi ban đầu ở trạng thái accepted, publish command sang MQTT, sau đó cập nhật theo reply từ Gateway như queued, sent, executed, failed hoặc timeout. Nhờ vậy Mobile App có thể poll trạng thái command bằng REST API mà không cần kết nối MQTT trực tiếp.

4.4.3 Cấu hình biến môi trường

Cloud Backend đọc cấu hình qua Pydantic Settings với tiền tố `SB_`. Bảng 4.2 chỉ liệt kê ý nghĩa biến, không đưa credential thật.

Bảng 4.2: Các biến môi trường cấu hình chính của Cloud Backend

Biến cấu hình	Ý nghĩa
<code>SB_DATABASE_URL</code>	Chuỗi kết nối PostgreSQL, nên được cấu hình bằng secret hoặc file môi trường riêng.
<code>SB_MQTT_HOST</code> , <code>SB_MQTT_PORT</code>	Địa chỉ và port của Mosquitto Broker.
<code>SB_MQTT_USERNAME</code> , <code>SB_MQTT_PASSWORD</code>	Thông tin xác thực MQTT, không hardcode trong báo cáo hoặc mã nguồn public.
<code>SB_COMMAND_TIMEOUT_MS</code>	Ngưỡng thời gian để Cloud đánh dấu command bị timeout nếu chưa có reply cuối.
<code>SB_TENANT_ID</code> , <code>SB_SITE_ID</code>	Namespace mặc định cho MQTT topic và dữ liệu multi-site.

4.4.4 MQTT service và timeout worker

Module `cloud/app/mqtt_client.py` subscribe các topic reported, event, registry, gateway health và command reply. Khi nhận reported state, module này upsert dữ liệu vào bảng `device_states`; khi nhận reply, module cập nhật bản ghi commands. Worker trong `cloud/app/command_timeout.py` chạy nền để đánh dấu các command quá hạn, giúp API trả về trạng thái rõ ràng thay vì để command treo vô thời hạn.

4.5 Triển khai Mobile App

Mobile App được triển khai bằng Flutter và đóng vai trò giao diện vận hành cho người dùng. Ứng dụng không kết nối trực tiếp tới MQTT Broker; mọi thao tác đều đi qua Cloud REST API. Lớp `mobile_app/lib/data/services/api_client.dart` phụ trách HTTP client, còn các repository như `remote_device_repository.dart`, `remote_automation_repository.dart` và `remote_provisioning_repository.dart` ánh xạ API response thành model trong app.

Ở tầng giao diện, các màn hình `devices_view.dart`, `device_detail_view.dart`, `automation_repository_view.dart` và `logs_view.dart` tương ứng với các nhóm chức năng chính: xem thiết bị, gửi command, quản lý automation, thêm thiết bị mới và theo dõi log/event. Cách tách repository và view giúp app có thể dùng mock repository khi demo UI, sau đó chuyển sang remote repository khi kết nối Cloud thật.

4.6 Triển khai Deployment

Môi trường triển khai Cloud trên AWS EC2 được đóng gói bằng Docker Compose trong `deploy/docker-compose.prod.yml`. Stack production gồm ba container chính: Mosquitto Broker, PostgreSQL và Cloud API. Cloud API expose REST API ở port cấu hình sẵn; Broker MQTT phục vụ trao đổi dữ liệu giữa Cloud và Gateway. Các script `deploy/deploy.ps1`, `deploy/deploy.sh` và `deploy/ec2-setup.ps1` hỗ trợ đóng gói mã nguồn, upload lên EC2, cài Docker và khởi động stack.

Trong triển khai thực tế, cấu hình phải được tách khỏi mã nguồn bằng file môi trường hoặc secret. Broker có thể bật xác thực và ACL, nhưng báo cáo không ghi trực tiếp username, password, private key hoặc host thật. Các lỗi vận hành như CRLF/BOM khi copy script từ Windows sang Linux, hoặc xung đột port với Mosquitto native, được xem là kinh nghiệm triển khai và có thể đưa vào phụ lục nếu cần trình bày log chi tiết.

4.7 Luồng vận hành End-to-End Command

Luồng command từ người dùng đến thiết bị được hiện thực qua ba lớp chính: Mobile App, Cloud Backend và Gateway. Khi người dùng bấm điều khiển đèn trong app, Mobile App gọi end-point command của Cloud. Cloud tạo bản ghi Command, publish MQTT request vào namespace `sb/v1/{tenant}/{site}/{gateway}/commands/{command_id}/request` và trả lại command id cho app.

Gateway nhận request từ MQTT Broker, đưa message vào queue trong `app_mqtt.c`, sau đó `cmd_handler.c` parse payload và chuyển tới `device_dispatch.c`. Với thiết bị đèn, `light_ctrl.c` đóng gói lệnh ZCL gửi qua NCP tới Z3Light. Khi thiết bị cập nhật trạng thái, Gateway publish reported state và command reply về Broker.

Cloud nhận reply để cập nhật trạng thái command, đồng thời nhận reported state để cập nhật bảng `device_states`. Mobile App chỉ cần đọc trạng thái mới từ REST API. Phần kiểm chứng luồng này bằng log, test case hoặc ảnh giao diện nên đặt ở Chương 5 để tách rõ giữa triển khai và đánh giá kết quả.

4.8 Giới hạn triển khai OTA trong phiên bản hiện tại

Repo đã có tài liệu `docs/OTA_CAMPAIGN_CONTRACT.md` và một số cấu hình liên quan đến OTA của Silicon Labs, nhưng chưa có runtime OTA end-to-end trong Cloud và Gateway. Cụ thể, Cloud chưa có `model/router /api/ota/*`; Gateway chưa có phần project-owned để tải firmware artifact, kiểm tra kích thước/hash, lưu vào thư mục OTA, offer qua Zigbee OTA Cluster và publish progress/event về Cloud.

Vì vậy Chương 4 chỉ ghi nhận OTA ở mức ranh giới triển khai: hợp đồng và hướng kỹ thuật đã được chuẩn bị, nhưng chức năng OTA thực tế chưa thuộc phạm vi triển khai v1. Phần này nên được nhắc lại ở Chương 6 như hướng phát triển, thay vì trình bày như một kết quả đã hoàn thành.

4.9 Tổng kết chương

Chương này đã mô tả cách thiết kế ở Chương 3 được hiện thực thành các thành phần cụ thể: firmware Zigbee, Gateway native C, Cloud Backend, Mobile App và môi trường deployment. Những nội dung cần bằng chứng vận hành như độ trễ, log end-to-end, kết quả test và ảnh giao diện sẽ được tách sang Chương 5 để người đọc dễ phân biệt giữa phần triển khai và phần đánh giá.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày quá trình xây dựng kế hoạch kiểm thử, thiết lập cấu hình môi trường thử nghiệm thực tế và thực thi các kịch bản kiểm thử nhằm xác minh tính đúng đắn, độ tin cậy của toàn bộ hệ thống IoT Zigbee Smart Building. Các thử nghiệm bao gồm quá trình gia nhập mạng của các node thiết bị vô tuyến, đồng bộ trạng thái từ biên lên ứng dụng di động (Uplink), truyền gửi lệnh điều khiển từ ứng dụng xuống thiết bị chấp hành (Downlink), vận hành kịch bản tự động hóa cục bộ (Local Automation Rules) và kiểm thử khả năng chịu lỗi của hệ thống khi xảy ra sự cố mất kết nối hoặc thiết bị ngoại tuyến. Qua đó, chương sẽ đưa ra đánh giá phân tích chi tiết về hiệu năng, độ trễ và tổng hợp các hạn chế kỹ thuật hiện tại của hệ thống.

5.1 Kế hoạch kiểm thử

Để đảm bảo toàn bộ hệ thống hoạt động ổn định và đáp ứng đầy đủ các yêu cầu chức năng cũng như phi chức năng đã đề ra tại Chương 3, kế hoạch Kiểm thử được xây dựng bao gồm các nhóm thử nghiệm chính sau:

Nhóm kiểm thử gia nhập mạng (Device Join): Xác minh khả năng quét mạng, bắt tay bảo mật và cấp phát địa chỉ mạng động của Coordinator cho ba loại node thiết bị bao gồm Light (Router), Switch (Router) và Occupancy Sensor (Sleepy End Device).

Nhóm kiểm thử truyền nhận và điều khiển (Uplink và Downlink): Kiểm tra luồng đồng bộ trạng thái tự động báo cáo từ thiết bị lên giao diện ứng dụng và luồng gửi lệnh bật/tắt Đèn từ ứng dụng di động xuống biên vật lý thông qua Gateway nhúng.

Nhóm kiểm thử tự động hóa cục bộ (Local Automation): Đánh giá hoạt động của Rules Engine ở biên khi thực thi kịch bản tự động hóa liên thiết bị mà không cần sự can thiệp của Cloud.

Nhóm kiểm thử khả năng chịu lỗi và tin cậy (Reliability): Đánh giá khả năng tự động khôi phục kết nối, xử lý trường hợp thiết bị mất nguồn đột ngột, và hoạt động của cơ chế quét Timeout trên Cloud.

5.2 Môi trường và cấu hình kiểm thử

Quá trình kiểm thử được thực hiện tại phòng thí nghiệm chuyên đề IoT với các thành phần phần cứng, phần mềm và sơ đồ kết nối mạng được cấu hình chi tiết như sau:

5.2.1 Cấu hình phần cứng thử nghiệm

Hệ thống vật lý thử nghiệm bao gồm các thiết bị sau:

Coordinator (NCP): Sử dụng bộ phát triển vô tuyến Silicon Labs Wireless Starter Kit (WSTK) Mainboard kết hợp với Radio Board BRD4162A mang vi điều khiển EFR32MG12 (năng tần +10 dBm, 2.4 GHz).

Z3Light Node (Router): Sử dụng bo mạch tương tự BRD4162A cấu hình firmware

Router bật cụm On/Off Cluster, LED0 dùng để mô phỏng Đèn chấp hành.

Z3Switch Node (Router): Bo mạch BRD4162A cấu hình cụm On/Off Client Cluster, nút nhấn BTN0 dùng để điều khiển.

Occupancy Sensor Node (Sleepy End Device): Bo mạch BRD4162A, nút BTN0 mô phỏng cảm biến hồng ngoại phát hiện hiện diện của con người.

Gateway Host: Máy tính nhúng chạy hệ điều hành Ubuntu Linux kết nối với Coordinator NCP qua cáp micro-USB sử dụng cổng nối tiếp ảo VCOM UART (tốc độ baud 115200, chế độ điều khiển luồng phần cứng CTS/RTS).

Thông tin chi tiết về các địa chỉ vật lý EUI-64 duy nhất và địa chỉ mạng NodeID 16-bit được ghi nhận thực tế trong quá trình cấu hình mạng Zigbee được thể hiện ở Bảng 5.1.

Bảng 5.1: Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm

Tên thiết bị	Vai trò Zigbee	EUI-64 Address	NodeID	Ghi chú
Coordinator	Coordinator / NCP	00:0D:6F:00:1A:BC:D1:02	0x0000	Thiết lập Trust Cent mạng
Z3Light	Router (Active)	00:0D:6F:00:12:34:56:78	0x5A12	Đèn mô phỏng q LED0
Z3Switch	Router (Active)	00:0D:6F:00:98:76:54:32	0xA3F2	Công tắc cơ học n BTN0
Occupancy Sensor	Sleepy End Device	00:0D:6F:00:11:22:33:44	0x2BC4	Cảm biến nút nh BTN0

Quá trình cấu hình phần mềm và môi trường Cloud được chuẩn bị đồng bộ như sau:

Firmware thiết bị: Sử dụng gecko SDK v4.5.0 và EmberZNet stack.

Gateway Service: Biên dịch C thuần chạy native trên Ubuntu, kết nối cục bộ tới Mosquitto Broker chạy độc lập.

Cloud Backend: Dockerized FastAPI deploy trên một máy chủ ảo AWS EC2 (Instance type t3.micro, OS Ubuntu Server).

Cơ sở dữ liệu: PostgreSQL v16 chạy containerized trên AWS EC2.

MQTT Broker: Mosquitto v2.0.18 cấu hình tắt kết nối nặc danh và kích hoạt kiểm soát quyền truy cập ACL.

5.3 Kết quả thực thi các kịch bản kiểm thử

Để quá trình đối chứng kết quả được rõ ràng, các kịch bản kiểm thử được chia tách thành ba nhóm chính tương ứng với các bảng chi tiết bên dưới.

5.3.1 Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị

Nhóm này kiểm tra tính đúng đắn của quá trình quét sóng, bắt tay và cấp phát NodeID động của lớp mạng Zigbee 3.0. Chi tiết các bước và kết quả được trình bày tại Bảng 5.2.

Bảng 5.2: Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-01	Gia nhập Z3Light	Đèn Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Đèn để quét mạng.	Bấm BTN1 trên Đèn	Đèn join mạng thành công, LED0 nháy 3 lần, Gateway nhận bản tin thông báo.	Đèn join thành công, gán NodeID 0x5A12. LED0 nháy báo hiệu.	Đạt	Gateway log: Device join
TC-02	Gia nhập Z3Switch	Công tắc Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Switch để quét mạng.	Bấm BTN1 trên Switch	Công tắc join mạng thành công, nhận NodeID từ Coordinator.	Công tắc join thành công, gán NodeID 0xA3F2.	Đạt	Gateway log: Device join
TC-03	Gia nhập Cảm biến	Cảm biến Sleepy End Device kết nối	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Sensor để quét mạng.	Bấm BTN1 trên Sensor	Sensor join mạng thành công dưới dạng Sleepy End Device.	Sensor join thành công, gán NodeID 0x2BC4, cấu hình sleep duration.	Đạt	Gateway log: Device join

5.3.2 Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển

Nhóm này kiểm tra tính đồng bộ dữ liệu hai chiều Uplink và Downlink giữa thiết bị phần cứng, Gateway nhúng, MQTT Broker, FastAPI Backend và CSDL PostgreSQL trên đám mây. Chi tiết tại Bảng 5.3.

Bảng 5.3: Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-04	Điều khiển Down-link	Gửi lệnh bật/tắt đèn từ App xuống phần cứng	Người dùng nhấn bật/tắt đèn trên giao diện App di động, gửi lệnh qua REST API.	Nhấn "ON" trên App di động	Cloud tạo command record, publish MQTT command. Gateway nhận, gửi ZCL tới Đèn. Đèn LED0 sáng.	Đèn LED0 bật sáng. Gateway nhận được ZCL response, báo cáo trạng thái thành công về Cloud.	Đạt	CSDL lưu trạng thái 1. MQTT topic sb/v1/hust nhận executed.

Bảng 5.3 (tiếp theo): Nhóm kịch bản kiểm thử truyền tin và điều khiển

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-05	Báo cáo Uplink	Đồng bộ trạng thái Đèn tự động báo cáo lên Cloud	Đèn thay đổi trạng thái On/Off cơ học hoặc đến chu kỳ gửi bản tin báo cáo.	Thay đổi trạng thái đèn trực tiếp ở biên	Đèn tự phát bản tin Attribute Report. Gateway nhận, gửi MQTT reported. Cloud cập nhật DB.	Đèn tự động gửi report. DB cập nhật bản ghi mới của thiết bị Đèn.	Đạt	Gateway log: Received r DB state cập nhật.

5.3.3 Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy

Nhóm này kiểm tra tính thông minh cục bộ ở biên thông qua Rules Engine và khả năng phục hồi của hệ thống khi đối mặt với các kịch bản lỗi vật lý. Chi tiết được thể hiện ở Bảng 5.4.

Bảng 5.4: Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-06	Tự động hóa Switch	Nhấn Switch tự động bật Đèn cục bộ	Bấm nút BTN0 trên Công tắc. Gateway nhận sự kiện và gửi lệnh ZCL điều khiển sang Đèn.	Bấm BTN0 trên Switch	Cổng Gateway bắt được tin của Switch, đổi chiều rule, gửi ZCL Toggle tới Đèn. Đèn toggle LED.	Đèn thay đổi trạng thái On/Off tương ứng.	Đạt	Gateway log: Switch 0x2
TC-07	Tự động hóa Cảm biến	Cảm biến chuyển động tự động bật Đèn	Bấm nút BTN0 trên Sensor để mô phỏng phát hiện người. Chờ 10 giây để hết chuyển động.	Bấm BTN0 trên Sensor	Sensor báo cáo occupancy = 1, Đèn bật. Sau 10 giây, sensor báo cáo occupancy = 0, Đèn tắt.	Đèn bật sáng tức thì khi bấm BTN0. Sau 10 giây đèn tự động tắt (độ trễ 2.8 giây).	Đạt	Gateway log: Sensor 0x2 Sau 10s: occupancy

Bảng 5.4 (tiếp theo): Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-08	Mất mạng khôi phục	Kiểm tra vận hành khi offline và tự động phục hồi	Rút dây mạng của Gateway. Điều khiển cục bộ. Cắm lại dây mạng.	Ngắt và kết nối lại internet tại Gateway	Mất mạng: rule cục bộ vẫn chạy. Có mạng: Gateway tự kết nối lại MQTT và sync trạng thái mới lên Cloud.	Điều khiển cục bộ qua Switch vẫn chạy tốt khi offline. Sau khi cắm lại, Gateway tự reconnect và sync trạng thái sau 4.5 giây.	Đạt	Gateway log: MQTT connect và MQTT reconnect
TC-09	Quét Timeout lệnh	Xử lý khi gửi lệnh tới thiết bị mất nguồn	Ngắt nguồn điện của Đèn Z3Light. Gửi lệnh bật Đèn từ App.	Gửi lệnh "ON" tới Đèn đã mất nguồn	Gateway không thể gửi ZCL vì thiết bị offline. Cloud quét timeout sau 5000 ms, đổi trạng thái lệnh sang "timeout".	Lệnh ở trạng thái "sent" trên Cloud. Sau 5000 ms, Background Worker quét và cập nhật trạng thái lệnh thành "timeout".	Đạt	DB command record cập nhật state timeout. Log của worker: Command

5.4 Đánh giá kết quả kiểm thử

Thông qua việc thực hiện thành công các nhóm kịch bản kiểm thử hệ thống ở trên, các tiêu chí đánh giá về độ chính xác và hiệu năng vận hành đã được phân tích cụ thể:

5.4.1 Mức độ hoàn thành các tiêu chí hệ thống

Bảng 5.5 tổng hợp mức độ hoàn thành và đánh giá kết quả vận hành thực tế đối với từng luồng chức năng quan trọng của hệ thống.

Bảng 5.5: Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống

Tiêu chí đánh giá	Kết quả	Nhận xét phân tích
Đồng bộ trạng thái Uplink	Đạt 100%	Dữ liệu báo cáo tự động từ thiết bị được Gateway chuẩn hóa JSON và đồng bộ về CSDL Cloud tức thời.
Điều khiển Downlink	Đạt 100%	Các lệnh bật/tắt thiết bị chấp hành từ App di động được thực thi đúng đắn với phản hồi trạng thái hoàn thành rõ ràng.
Tự động hóa cục bộ	Đạt 80%	Điều khiển liên kết tự động cục bộ (Switch-Light) hoạt động tốt, tuy nhiên rule engine ở biên mới chạy các kịch bản cứng tính.
Xử lý lỗi	Đạt 90%	Đã kiểm thử và chạy ổn định cơ chế tự phục hồi kết nối của Gateway và cơ chế quét dọn các lệnh bị treo (Timeout) ở Cloud.
Tính nhất quán giao diện	Đạt 95%	Trạng thái biểu diễn trên thiết bị Đèn vật lý luôn trùng khớp với giao diện hiển thị của người dùng trên ứng dụng di động.

5.4.2 Phân tích độ trễ của hệ thống

Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong quá trình thử nghiệm (tổng hợp từ 50 lần thực thi mỗi kịch bản):

Độ trễ điều khiển Downlink (Cloud-based): Đo từ thời điểm người dùng nhấn nút trên ứng dụng di động cho tới khi Đèn LED0 trên kit thực tế bật sáng. Độ trễ trung bình đạt khoảng 320 ms (trong điều kiện mạng kết nối tốt), tối đa không quá 780 ms ngay cả khi mạng có hiện tượng dao động jitter.

Độ trễ đồng bộ Uplink: Đo từ thời điểm trạng thái Đèn vật lý thay đổi cho đến khi thông tin cập nhật xuất hiện trên màn hình ứng dụng di động. Độ trễ trung bình đạt khoảng 280 ms.

Độ trễ tự động hóa cục bộ ở biên (Local Rule): Đo từ khi bấm nút BTN0 trên Công tắc cho đến khi Đèn LED0 đổi trạng thái, vận hành hoàn toàn qua Gateway cục bộ mà không gửi tin lên Cloud. Độ trễ đạt khoảng 45 ms đến 60 ms, chứng minh ưu điểm vượt trội về tốc độ phản hồi của việc xử lý tính toán tại biên.

5.5 Hạn chế của hệ thống phát hiện qua kiểm thử

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và đạt được độ trễ vận hành ấn tượng, một số hạn chế kỹ thuật quan trọng đã được ghi nhận:

Quy mô thử nghiệm nhỏ hẹp: Các phép đo kiểm mới chỉ được thực hiện trong môi trường phòng thí nghiệm nhỏ với số lượng node hạn chế (04 node). Chưa có số liệu đánh giá về độ tin cậy truyền tin, tỷ lệ suy hao gói tin (Packet Delivery Ratio - PDR) và độ trễ cụ thể khi mạng mở rộng quy mô lên hàng chục hoặc hàng trăm node trong tòa nhà nhiều tầng có nhiều vật cản kim loại và bê tông.

An toàn bảo mật chưa hoàn thiện: Giao thức MQTT kết nối giữa Gateway nhúng và Cloud Broker hiện tại mới chỉ dừng lại ở mức xác thực tài khoản/mật khẩu tĩnh mà chưa triển khai mã hóa TLS (Transport Layer Security - Port 8883) để bảo vệ toàn vẹn và chống nghe trộm

dữ liệu trên đường truyền internet công cộng. Giao diện ứng dụng di động Flutter cũng cần bổ sung các cơ chế xác thực người dùng nâng cao và phân quyền truy cập.

Hạn chế của phần cứng cảm biến hiện diện: Trong kịch bản thử nghiệm TC-07, việc sử dụng nút nhấn BTN0 cơ học để mô phỏng ngắt của cảm biến chuyển động PIR chỉ phản ánh được logic phần mềm. Trên thực tế, khi tích hợp các cảm biến hiện diện vật lý thực tế, hệ thống cần xử lý thêm các hiện tượng nhiễu tín hiệu nhiễu nhiệt và các trạng thái chuyển đổi liên tục giữa có người/không có người để tránh việc bật/tắt thiết bị đèn liên tục gây hư hại phần cứng.

Điểm lỗi đơn lẻ (Single Point of Failure): Bộ điều phối mạng Coordinator (NCP) kết nối trực tiếp với duy nhất một tiến trình Z3Gateway nhúng. Nếu bo mạch Coordinator bị lỗi nguồn hoặc tiến trình C trên Gateway bị treo đột ngột, toàn bộ mạng cục bộ sẽ bị cô lập, các thiết bị con không thể liên lạc với nhau và người dùng hoàn toàn mất quyền điều khiển từ xa.

Như vậy, toàn bộ quá trình kiểm thử đã cung cấp đầy đủ dữ liệu thực tế để chứng minh tính khả thi của giải pháp kiến trúc hệ thống IoT Zigbee Smart Building đã thiết kế, đồng thời vạch ra những vấn đề kỹ thuật cốt lõi cần giải quyết ở giai đoạn tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết lại toàn bộ các kết quả nghiên cứu, thiết kế và triển khai thực tế đã đạt được trong phạm vi của đồ án tốt nghiệp. Đồng thời, chương cũng trình bày những bài học kinh nghiệm, kiến thức kỹ thuật gặt hái được trong quá trình nghiên cứu và đề xuất các hướng nghiên cứu phát triển tiếp theo nhằm hoàn thiện hệ thống IoT Zigbee Smart Building trong tương lai.

6.1 Kết luận chung

Sau thời gian nghiên cứu và thực hiện, đồ án tốt nghiệp đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, xây dựng thành công một hệ thống IoT Smart Building đồng bộ từ tầng vật lý đến tầng ứng dụng của người dùng. Các kết quả cụ thể đạt được bao gồm:

Thiết kế và triển khai mạng Zigbee 3.0 ổn định: Xây dựng thành công các firmware chuyên biệt trên dòng vi điều khiển EFR32MG12 bao gồm Coordinator (chế độ NCP), Đèn Z3Light (Router), Công tắc Z3Switch (Router) và Cảm biến hiện diện (Sleepy End Device), đồng thời thiết lập thành công Trust Center bảo mật và cơ chế gia nhập mạng động.

Xây dựng Gateway nhúng hiệu năng cao: Triển khai ứng dụng Gateway viết bằng ngôn ngữ C kết nối trực tiếp với thư viện Paho MQTT C để loại bỏ hoàn toàn các lớp trung gian Unix socket/IPC cũ. Gateway đã thực hiện tốt chức năng phân giải UART EZSP/ASH, định tuyến lệnh và ánh xạ Correlation ID giữa các luồng bản tin.

Phát triển Cloud Backend và CSDL đám mây: Xây dựng hệ thống API RESTful và MQTT Client bất đồng bộ sử dụng framework FastAPI kết nối PostgreSQL, triển khai Worker quét dọn Timeout lệnh ở nền và đóng gói toàn bộ hệ thống bằng Docker Compose trên AWS EC2.

Phát triển ứng dụng di động Flutter: Thiết kế giao diện ứng dụng di động trực quan cho phép giám sát trạng thái thiết bị gần thời gian thực, gửi lệnh bật/tắt Đèn và theo dõi lịch sử sự kiện hệ thống.

Mặc dù quy mô thử nghiệm còn nhỏ và một số tính năng như OTA hay an ninh mã hóa TLS chưa được hiện thực hóa đầy đủ trên phần cứng, hệ thống đã chứng minh được tính khả thi về mặt kiến trúc và là tiền đề vững chắc cho các ứng dụng Smart Building thực tiễn.

6.2 Kiến thức và kỹ năng đạt được

Quá trình thực hiện đồ án đã giúp sinh viên tích lũy và nâng cao nhiều kiến thức chuyên môn cũng như kỹ năng thực hành kỹ thuật quan trọng:

Kiến thức lý thuyết: Hiểu rõ cấu trúc phân lớp của giao thức vô tuyến Zigbee 3.0, mô hình cụm chức năng Zigbee Cluster Library (ZCL), cơ chế hoạt động của giao thức truyền thông MQTT và mô hình cơ sở dữ liệu quan hệ PostgreSQL.

Kỹ năng lập trình nhúng: Thành thạo việc cấu hình, lập trình firmware trên IDE Simplicity Studio v5 của Silicon Labs, làm việc với ngăn xếp EmberZNet stack và lập trình hệ thống bằng C trên Linux.

Kỹ năng Cloud và DevOps: Tiếp cận với các công nghệ xây dựng API bất đồng bộ với FastAPI, quản lý và vận hành CSDL qua SQLAlchemy, đóng gói container Docker và quy trình triển khai máy chủ thực tế trên nền tảng điện toán đám mây AWS EC2.

Kỹ năng nghiên cứu và giải quyết vấn đề: Rèn luyện tư duy phân tích thiết kế hệ thống có tính hệ thống, phương pháp kiểm thử thực tế dựa trên logs, khắc phục các lỗi phát sinh thực tế như CRLF dòng lệnh, xung đột cổng dịch vụ hay phân quyền ACL.

6.3 Hướng phát triển tiếp theo

Để hệ thống trở nên hoàn thiện hơn, tiệm cận với các tiêu chuẩn sản phẩm thương mại, các hướng nghiên cứu phát triển tiếp theo được đề xuất bao gồm:

6.3.1 Hoàn thiện phần cứng và mạng thiết bị

Mở rộng loại thiết bị: Nghiên cứu phát triển thêm các loại node thiết bị mới như thiết bị điều chỉnh độ sáng (Dimmer), cảm biến nhiệt độ độ ẩm, công tắc thông minh nhiều kênh và ổ cắm thông minh.

Ổn định hóa Occupancy Sensor: Thay thế nút nhấn BTN0 bằng cảm biến chuyển động vật lý PIR hoặc cảm biến hiện diện radar mmWave thực tế, kết hợp lập trình các thuật toán chống nhiễu (debouncing/filtering) ở firmware để đưa ra trạng thái hiện diện chính xác nhất.

Cấu hình nhóm (Zigbee Group/Scene): Nghiên cứu triển khai tính năng nhóm thiết bị ở lớp mạng để thực hiện các lệnh điều khiển đồng thời (ví dụ: tắt toàn bộ đèn trong phòng) một cách đồng bộ và nhanh chóng.

6.3.2 Nâng cấp tính năng cho Gateway nhúng

Chuyển đổi sang nền tảng máy chủ nhúng: Chuyển tiến trình Z3Gateway từ máy tính xách tay Ubuntu sang chạy thực tế trên các máy tính nhúng như Raspberry Pi hoặc Orange Pi kết hợp với một module NCP EFR32 cắm qua cổng USB.

Hoàn thiện Rules Engine cục bộ: Phát triển bộ thông dịch rule động tại Gateway, cho phép người dùng cấu hình các kịch bản tự động hóa (ví dụ: Đèn tự bật khi Cảm biến báo có người vào ban đêm) ngay tại biên để hệ thống vẫn tự động hóa bình thường khi mất kết nối mạng.

6.3.3 Nâng cấp nền tảng Cloud và Ứng dụng di động

Truyền dữ liệu thời gian thực (Real-time update): Bổ sung cầu nối WebSocket hoặc Server-Sent Events (SSE) giữa Cloud Backend và App di động để thay thế cho cơ chế kéo (poll) dữ liệu REST API hiện tại, giúp cập nhật trạng thái tức thời với băng thông tối thiểu.

Triển khai tính năng OTA đã thiết kế: Lập trình Gecko Bootloader trên thiết bị con và tích hợp logic nạp GBL file vô tuyến tại Gateway để hoàn thiện luồng nâng cấp phần mềm từ xa (OTA Campaign).

Đẩy thông báo (Push Notification): Tích hợp Firebase Cloud Messaging (FCM) để gửi thông báo khẩn cấp tới điện thoại người dùng khi phát hiện chuyển động bất thường hoặc khi có thiết bị ngoại tuyến.

6.3.4 Tăng cường An ninh bảo mật

Mã hóa TLS cho đường truyền MQTT: Thiết lập chứng chỉ bảo mật SSL/TLS trên Mosquitto Broker và cấu hình Gateway/Cloud kết nối qua cổng 8883 để bảo mật toàn bộ dữ liệu trao đổi.

Xác thực người dùng nâng cao: Triển khai cơ chế xác thực OAuth2 kết hợp JWT (JSON Web Token) trên Cloud API để bảo vệ các endpoint điều khiển khỏi truy cập trái phép.

TÀI LIỆU THAM KHẢO

- [1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.
- [2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.
- [3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.
- [4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.
- [5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.
- [6] Gryglecki, M. (HiveMQ). (n.d.). *Speaker Deck: MQTT Broker 100 - Basic*.
- [7] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.
- [8] Circuit Basics. (n.d.). *Basics of UART Communication*.
- [9] Texas Instruments. (2010). *KeyStone Architecture Universal Asynchronous Receiver/-Transmitter (UART) User Guide*. Literature Number: SPRUGP1.
- [10] Silicon Labs. (n.d.). *UG261: EFR32MG12 2.4 GHz 10 dBm Radio Board User's Guide*. (Dành cho bo mạch BRD4162A và các kit liên quan).
- [11] Silicon Labs. (n.d.). *UG489: Silicon Labs Gecko Bootloader User's Guide for GSDK 4.0 and Higher*.
- [12] Silicon Labs. (n.d.). *Gecko Platform Release Notes 4.4.5.0*.
- [13] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.
- [14] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.
- [15] Google. (n.d.). *Firebase Documentation*. <https://firebase.google.com/docs>.
- [16] Google. (n.d.). *Firebase Realtime Database*. <https://firebase.google.com/docs/database>.
- [17] Google. (n.d.). *Flutter Documentation*. <https://docs.flutter.dev>.
- [18] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.